

2.5.2 Accessibility and Usability

When including inputs, it is an accessibility requirement to add labels alongside. This is needed so those who use assistive technologies can tell what the input is for. Also, clicking or touching a label gives focus to the label's associated form control. This improves the accessibility and usability for sighted users, increases the area a user can click or touch to activate the form control. This is especially useful (and even needed) for radio buttons and checkboxes, which are tiny.

See above example under 2.4 and 2.5 how to associate the **<label>** with an **<input>** element.

2.6 What is Git and why use it?, Git setup and config (git config), Creating a local repository: git init

2.6.0 What is Git and why do we use it?

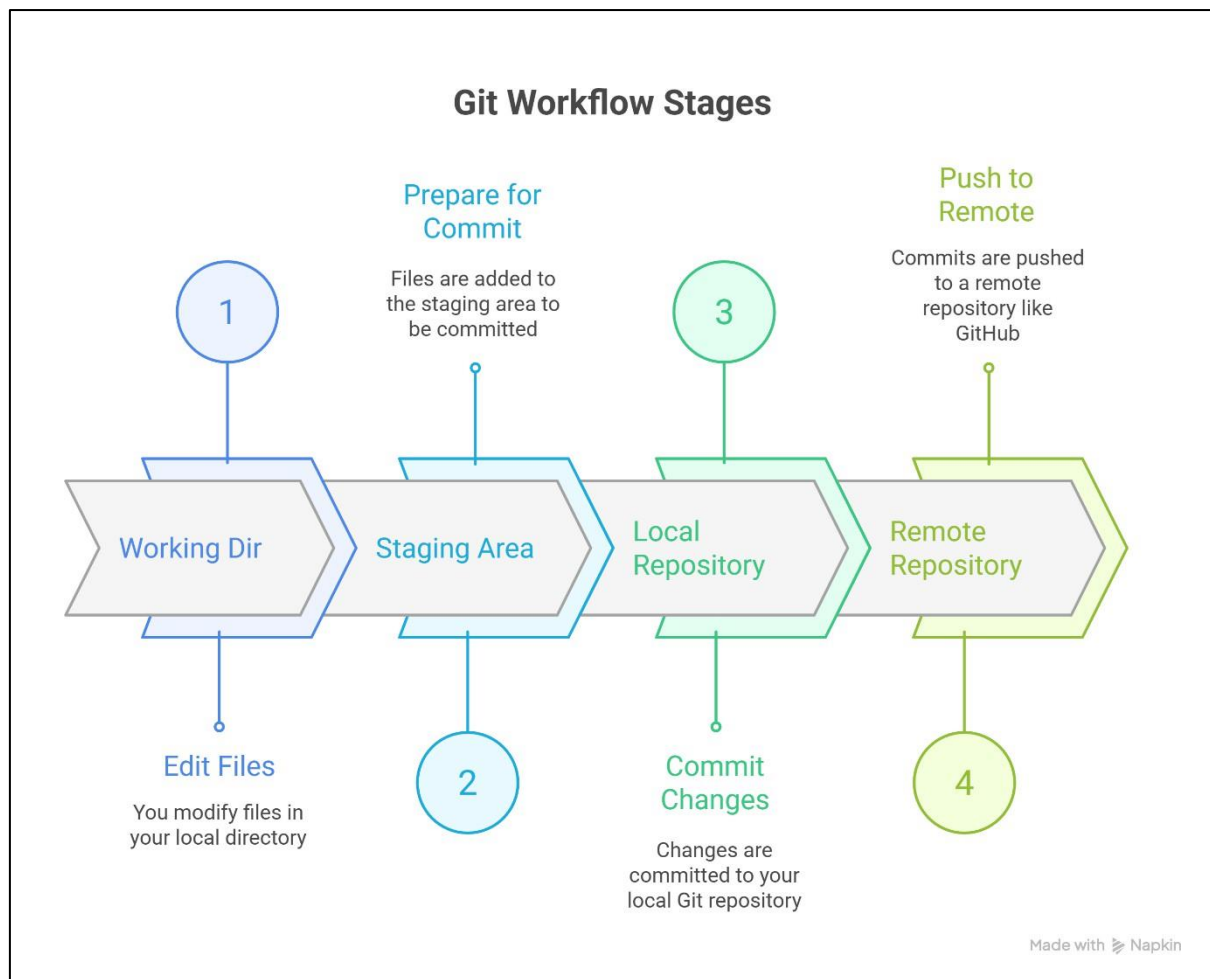
- Git is a **Version Control System (VCS)**, means Every developer has a full copy of repo.
- It helps keep track of every change you make to files in a project.
- If something breaks, you can go back to an earlier version.
- Multiple people can work on the same project without disturbing each other's work.
- Git is widely used in the software industry to manage projects and teamwork.

Key points:

- Git runs on your local computer.
- GitHub is a website (cloud) where Git projects can be stored and shared.
- Together, Git + GitHub = teamwork + backup + tracking changes.

Git has four important areas:

1. **Working Directory** – Where you actually edit files (your project folder).
2. **Staging Area (Index)** – A temporary area where you put the changes you want to commit.
3. **Local Repository** – The permanent history of commits stored on your computer.
4. **Remote Repository (GitHub/GitLab/Bitbucket)** – The copy of your repository stored on the server for sharing and collaboration.



Quick check: In VS Code, open the Terminal and run:

`git --version`

If you see a version (e.g., git version 2.45.x), you're set.

2.6.1 First-time Git setup in VS Code (one-time)

1. Open VS Code → **Terminal** → **New Terminal**.
2. Set your identity (appears in commit history):

Creating a GitHub Account

1. Go to <https://github.com/>
2. Sign up with email → verify → choose username.
3. Create a new repository:
 - Repository Name: web-project
 - Description: " Learn Git from Scratch !! "
 - Public → Add README.md → Create Repository.

github.com/new

Updated Fortinet N... how to give self intr... PB Academy GATE CS Preparatio... Master Course : Agil... AWS Certified Soluti... SOLUTION LINKS F... All Bookmarks

New repository

Type / to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * Firstyearpranshivarma-Educator Repository name * web-project
web-project is available.

Great repository names are short and memorable. How about [ideal-octo-enigma](#)?

Description Learn Git from Scratch !!
25 / 350 characters

2 Configuration

Choose visibility * Public
Choose who can see and commit to this repository

github.com/new

Updated Fortinet N... how to give self intr... PB Academy GATE CS Preparatio... Master Course : Agil... AWS Certified Soluti... SOLUTION LINKS F... All Bookmarks

Great repository names are short and memorable. How about [ideal-octo-enigma](#)?

Description Learn Git from Scratch !!
25 / 350 characters

2 Configuration

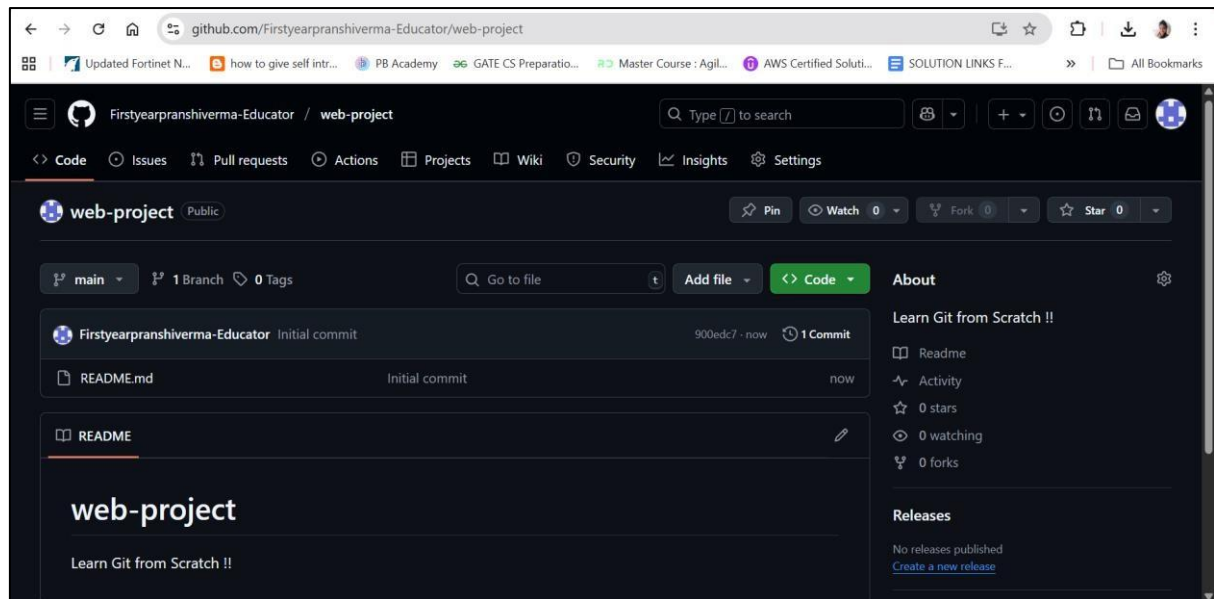
Choose visibility * Public
Choose who can see and commit to this repository

Add README On
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

Create repository



We will use this GitHub Repo to push our project code remotely after few steps.

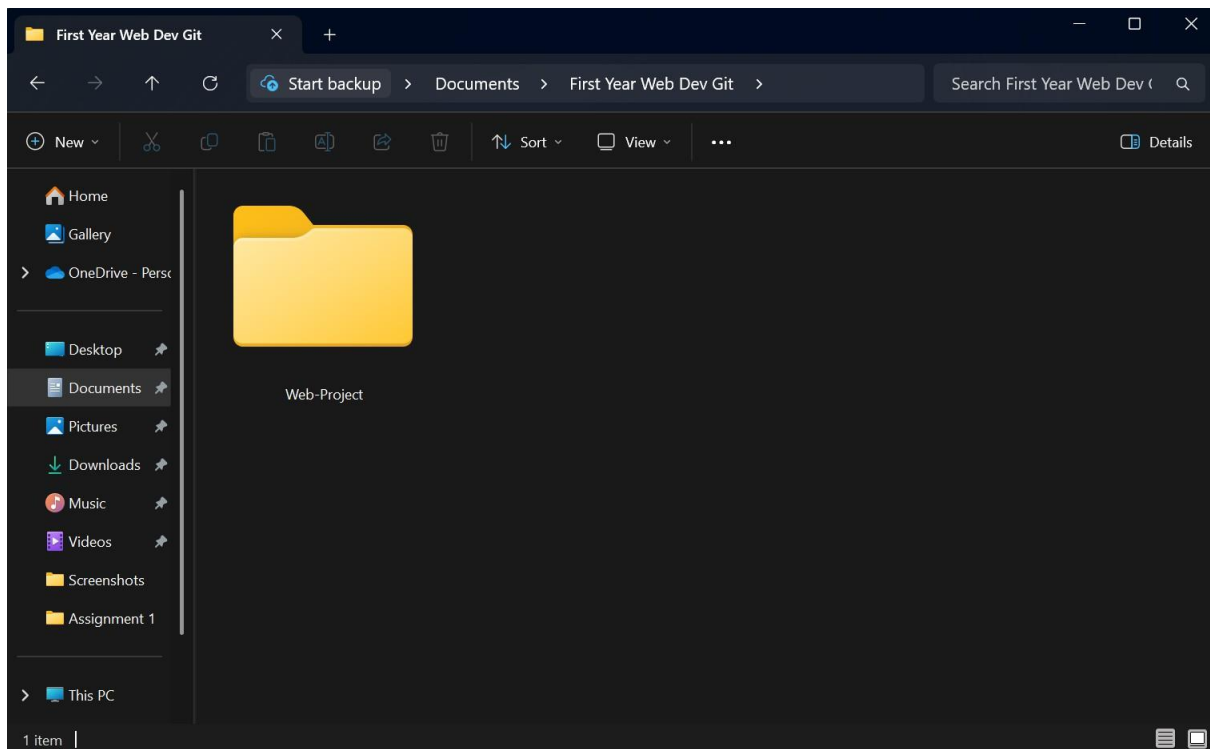
2.6.2 Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

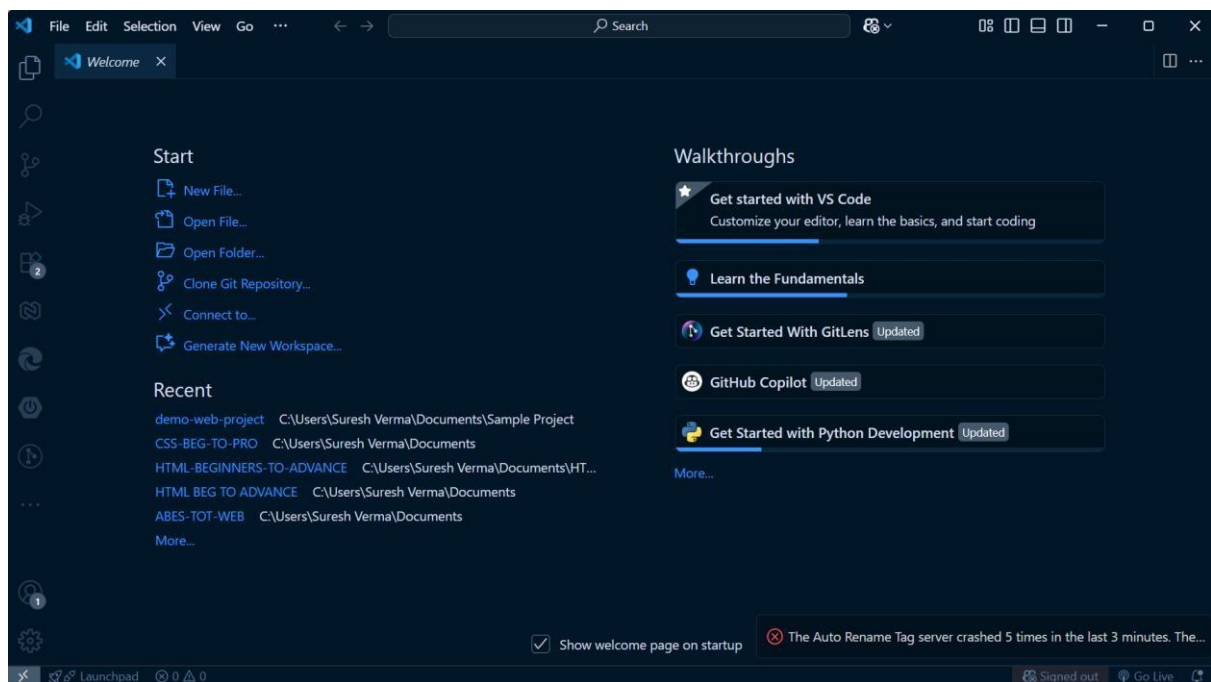
Using VS Code (Beginner-Friendly UI)

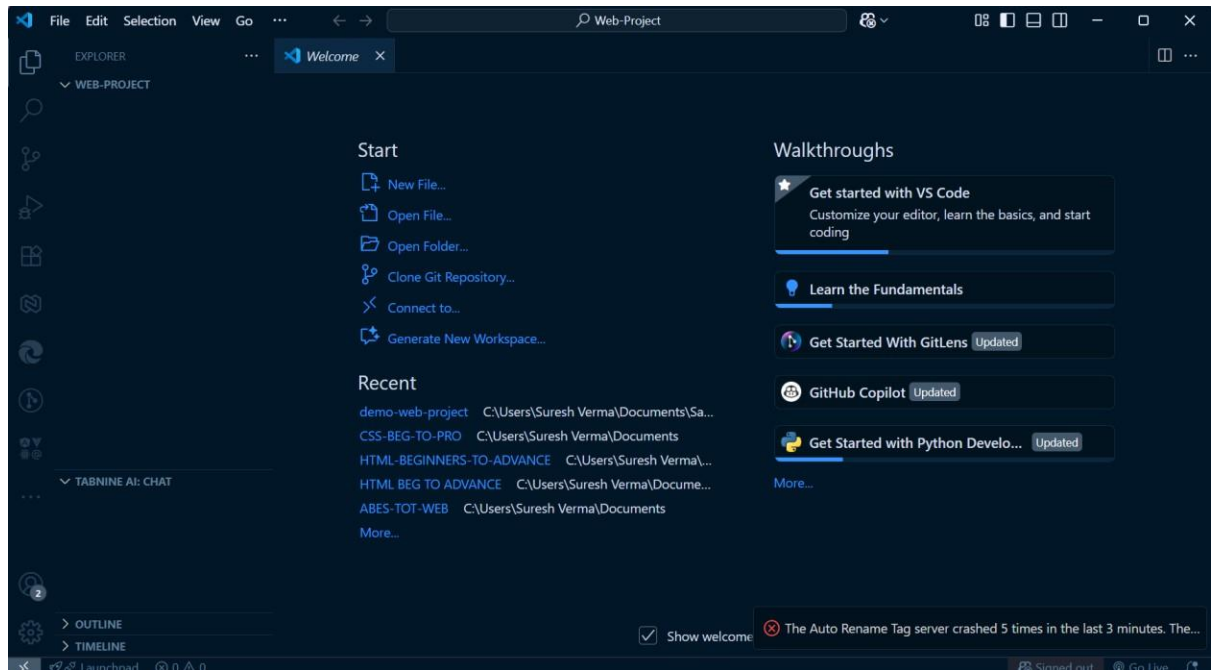
Create a project folder

Example: web-project



- Open it in VS Code (**File** → **Open Folder**).





Create project files

Example: index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<title>My First GitHub Project</title>
```

```
</head>
```

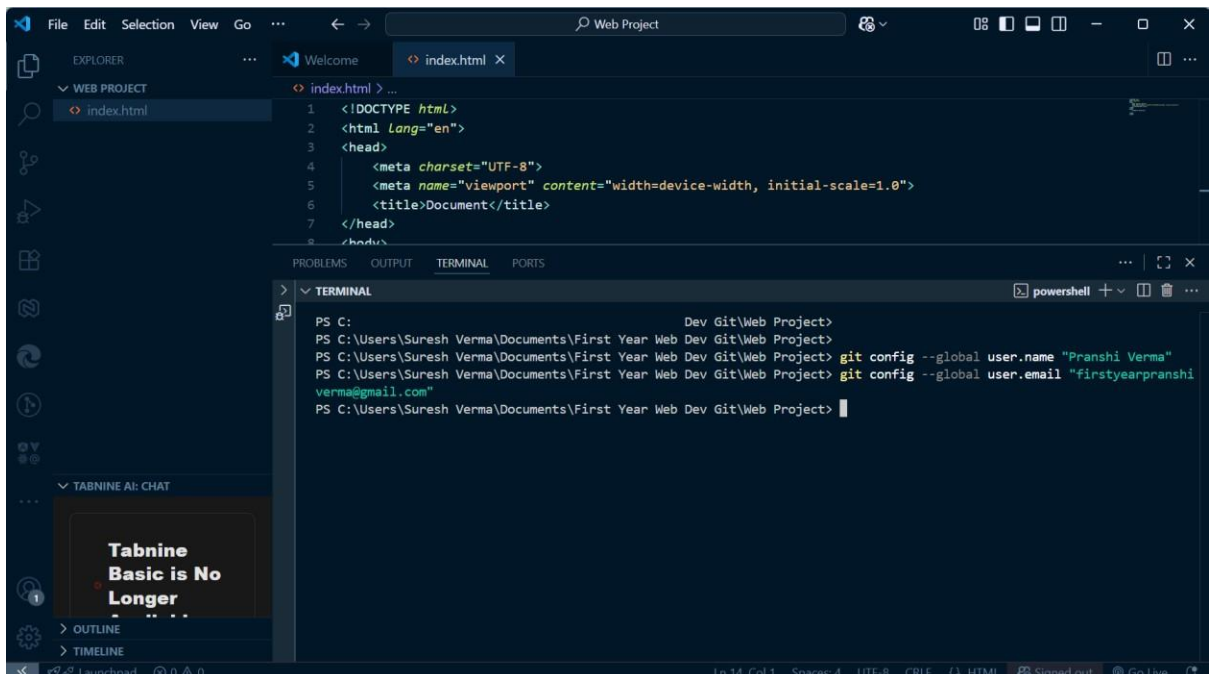
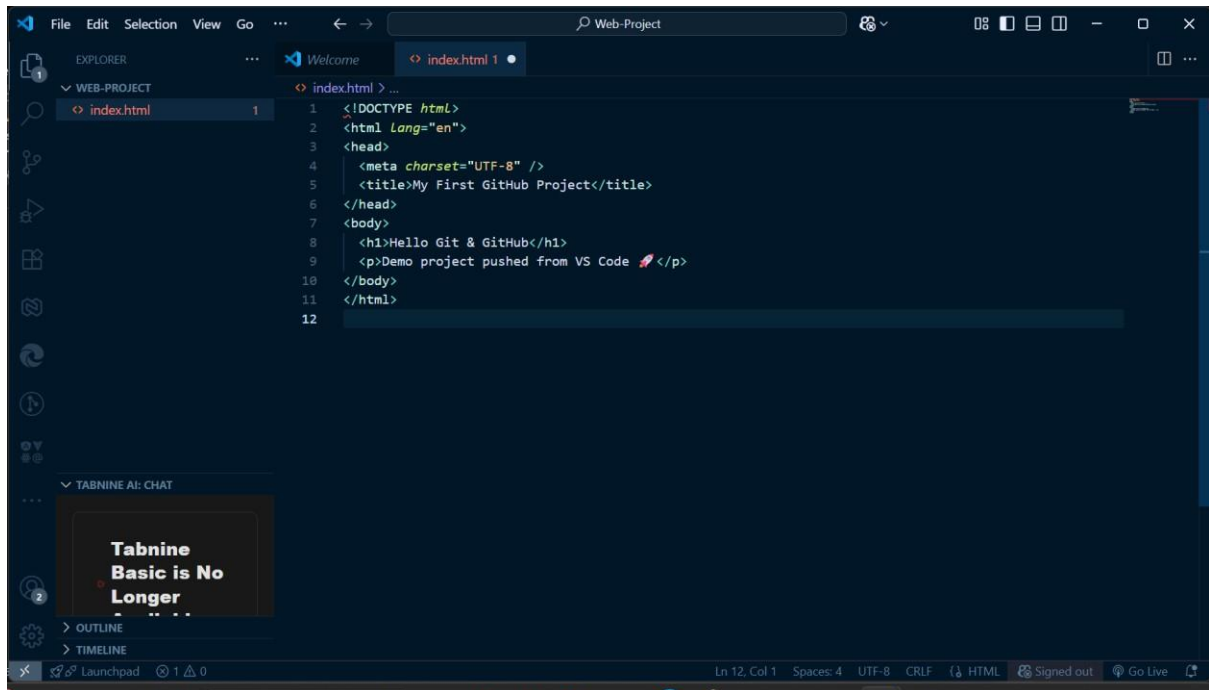
```
<body>
```

```
<h1>Hello Git & GitHub</h1>
```

<p>Demo project pushed from VS Code </p>

</body>

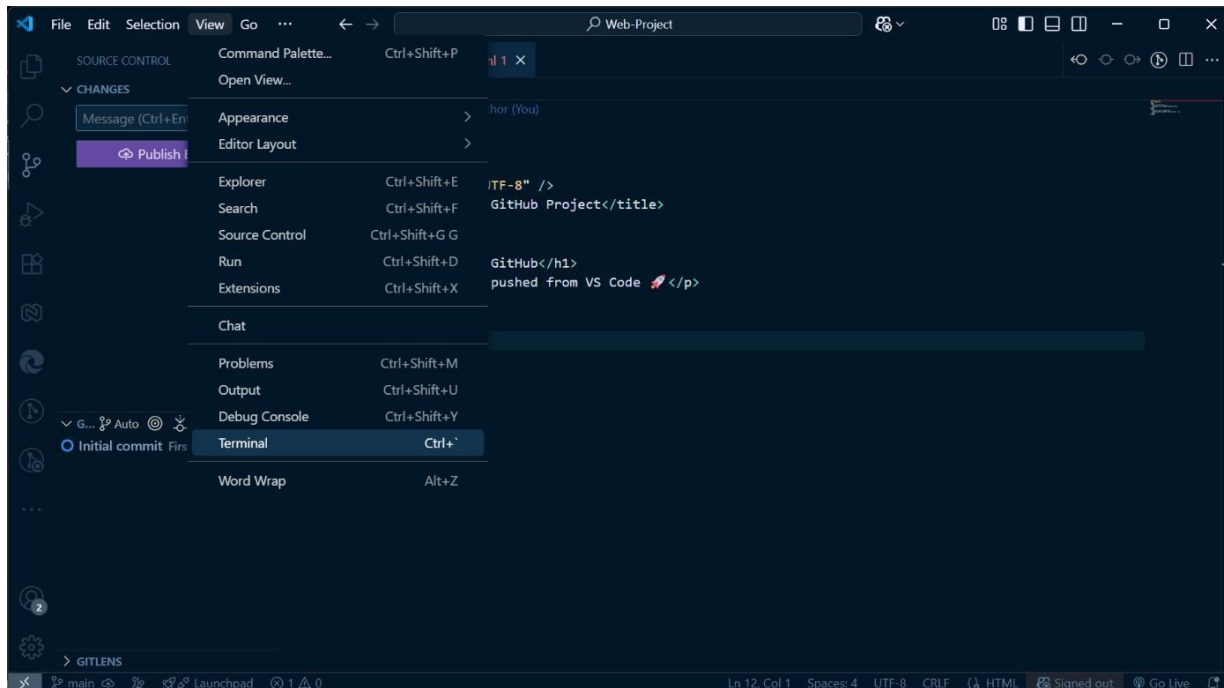
</html>



Initialize Git (create local repository)

- Where to Perform This Step?

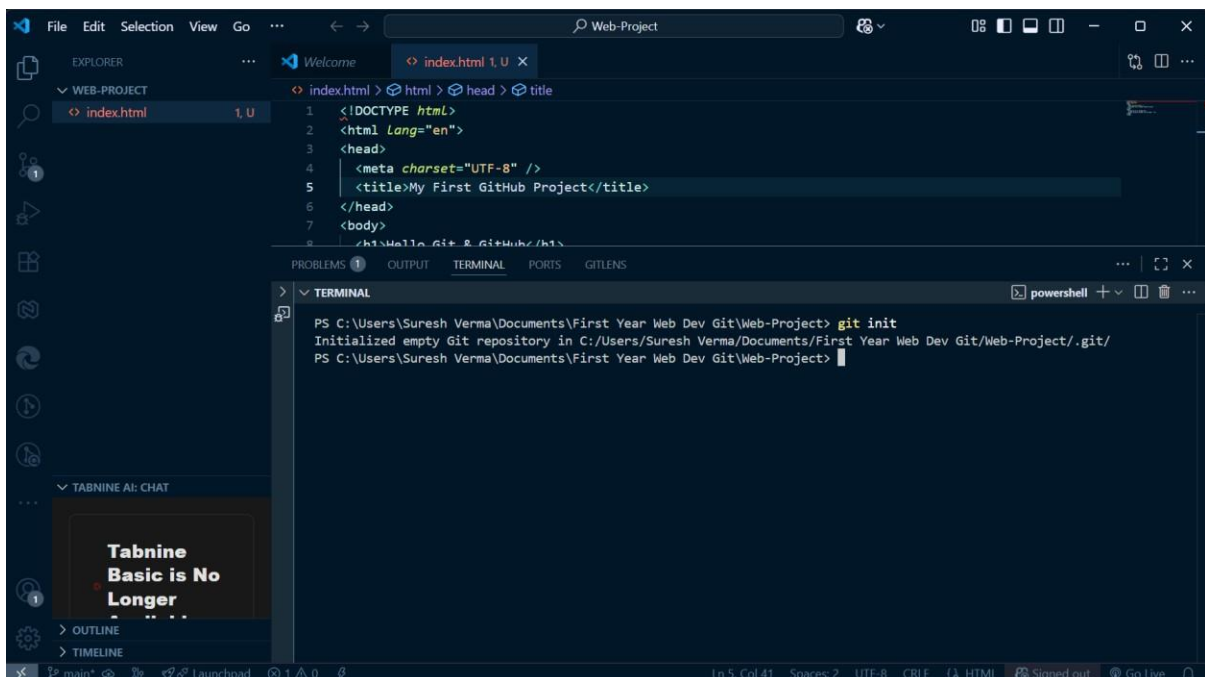
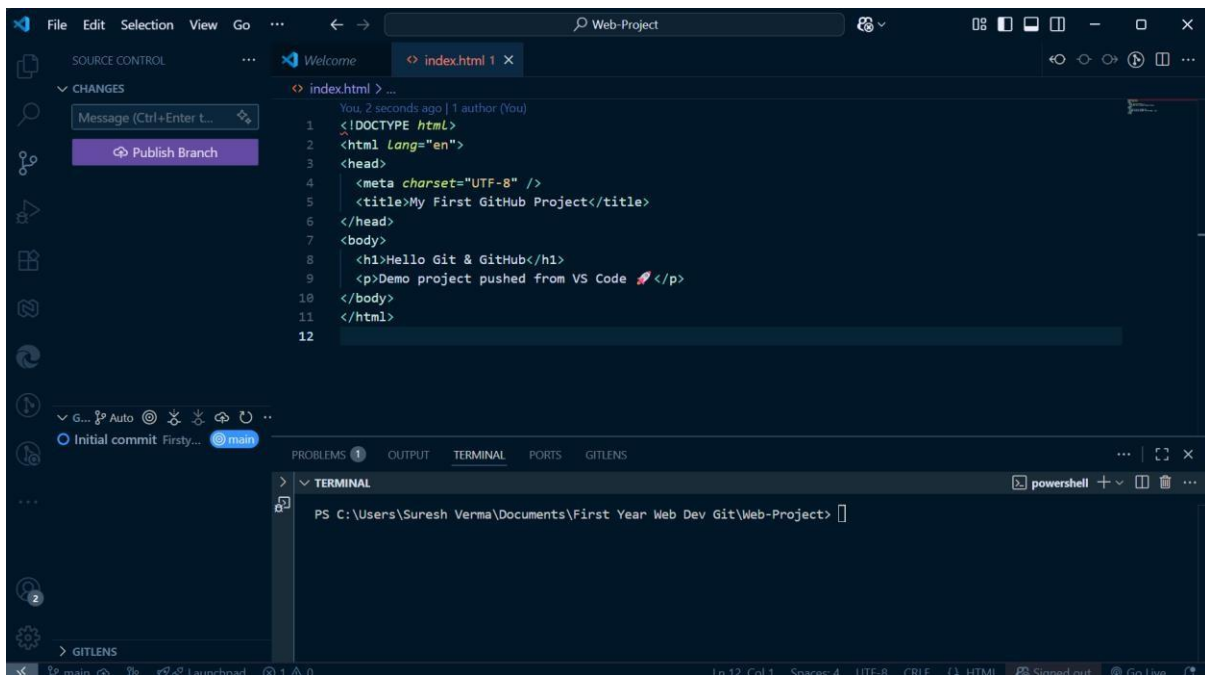
- This step is performed **in your terminal/command prompt (VS Code terminal or Git Bash)** — not on GitHub’s website.
- **In terminal:**



Use the commands:

Initialize the repository:

Using the command `git init`



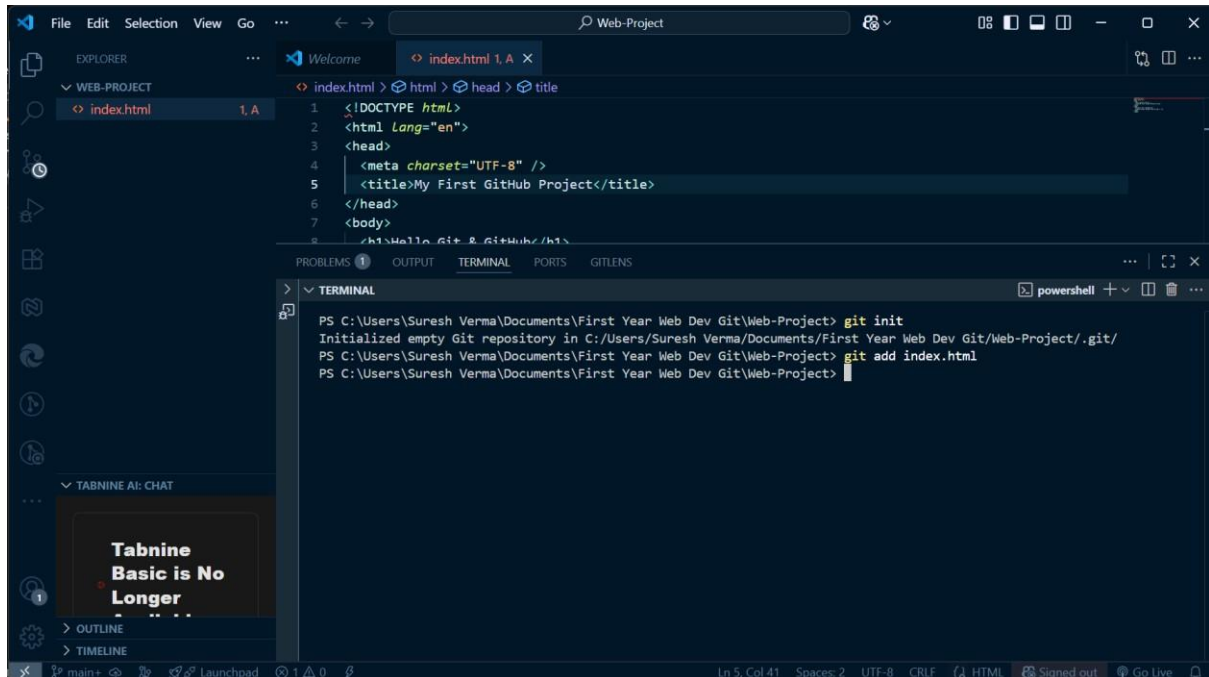
- This command creates a **hidden .git folder**, which means Git is now tracking your project.
- Add and commit files:

2.7 Tracking changes: add, commit, status, log,.gitignore basics,Practical

2.7.0 Add a Single File

If you only want to add **one file** to the staging area (for example, index.html):

`git add index.html`



Add Multiple Specific Files

You can also add more than one file at a time by naming them:

`git add index.html style.css`

Add All Files (Recommended when you want to add everything)

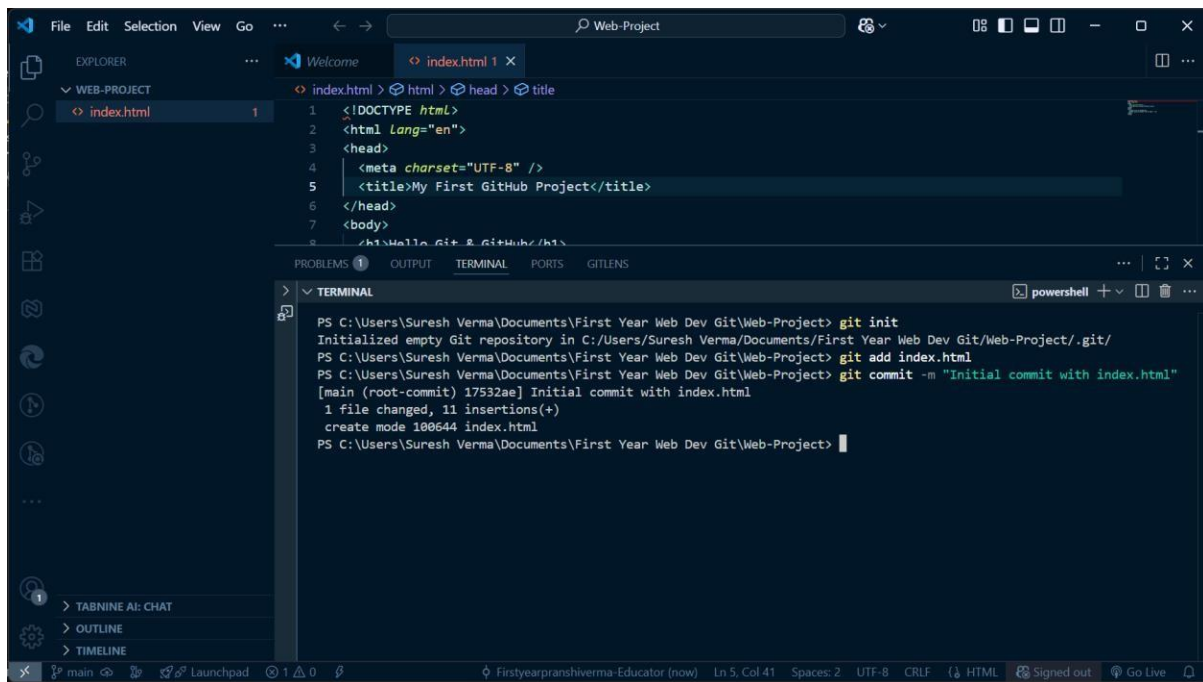
If you want to add **all files in the project folder**:

`git add .`

2.7.1 Commit the Changes

After adding files, you need to **commit** them with a message:

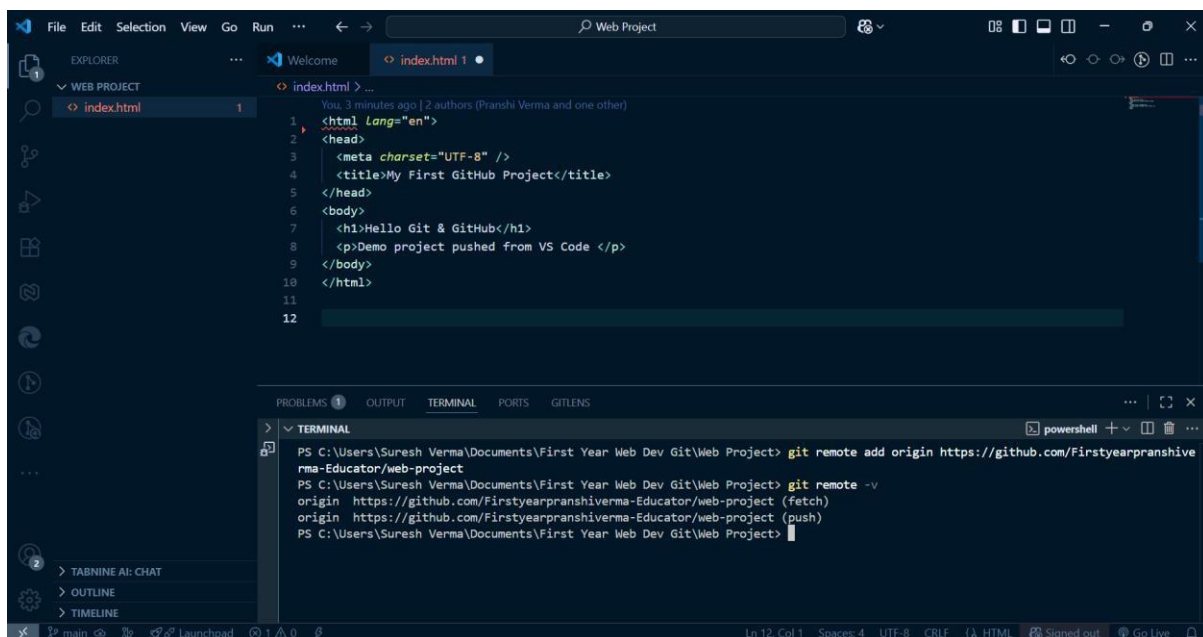
`git commit -m "Initial commit with index.html"`



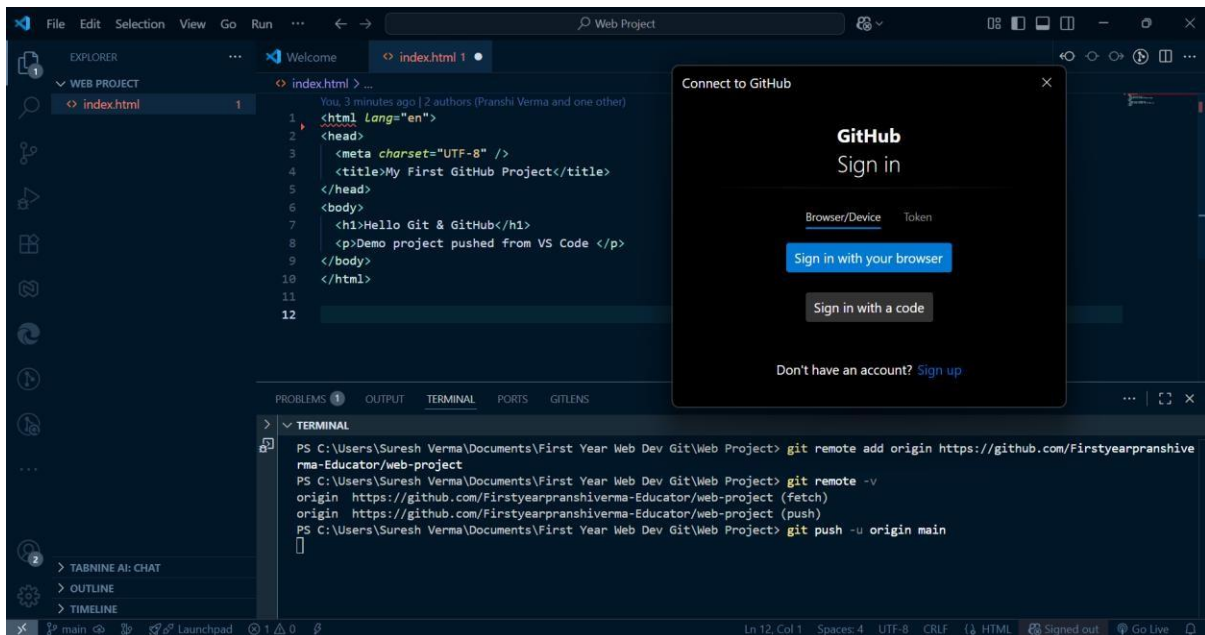
```
File Edit Selection View Go ... Web-Project
EXPLORER
  WEB-PROJECT
    index.html 1
index.html 1
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <title>My First GitHub Project</title>
6 </head>
7 <body>
8   <h1>Hello Git & GitHub</h1>
9 </body>
10 </html>
11
12
PROBLEMS 1 OUTPUT TERMINAL PORTS GITLENS
TERMINAL
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git init
Initialized empty Git repository in C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project\.git\
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git add index.html
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git commit -m "Initial commit with index.html"
[main (root-commit) 17532ae] Initial commit with index.html
1 file changed, 11 insertions(+)
create mode 100644 index.html
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project>
```

A **commit** is like saving a snapshot of your project at that point in time.

Now, project file is saved in local repository.

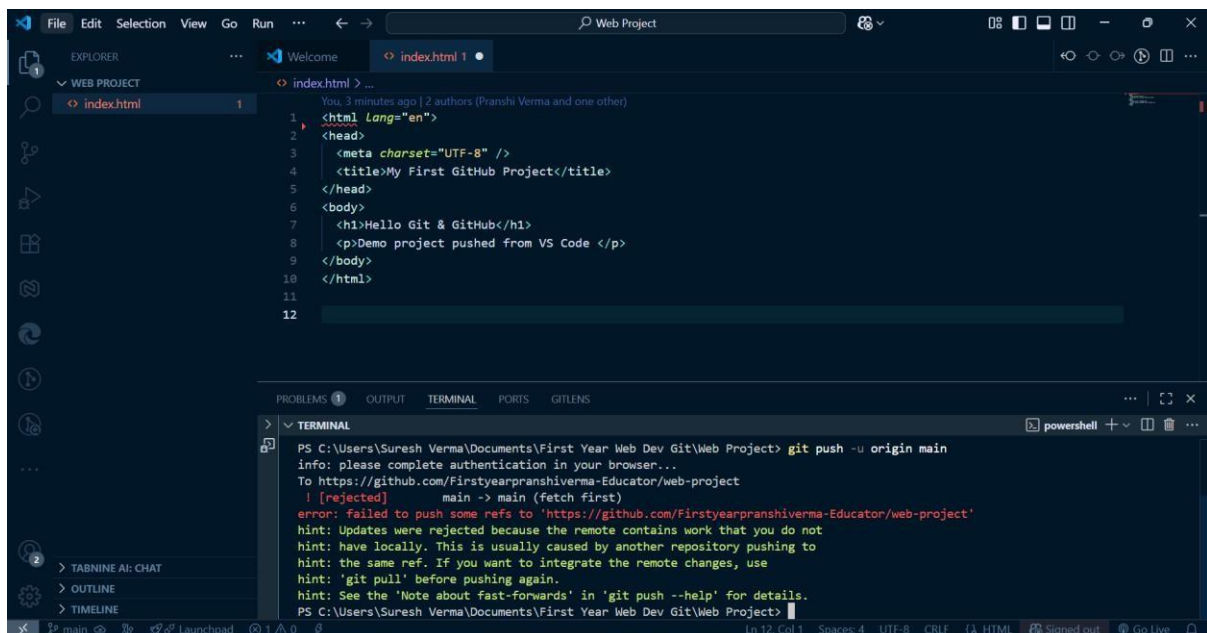


```
File Edit Selection View Go Run ... Web-Project
EXPLORER
  WEB-PROJECT
    index.html 1
index.html 1
1 You, 3 minutes ago [2 authors (Pranshi Verma and one other)]
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <title>My First GitHub Project</title>
6 </head>
7 <body>
8   <h1>Hello Git & GitHub</h1>
9   <p>Demo project pushed from VS Code</p>
10 </body>
11 </html>
12
PROBLEMS 1 OUTPUT TERMINAL PORTS GITLENS
TERMINAL
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git remote add origin https://github.com/Firstyearpranshive
rma-Educator/web-project
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project> git remote -v
origin https://github.com/Firstyearpranshive-Educator/web-project (fetch)
origin https://github.com/Firstyearpranshive-Educator/web-project (push)
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web-Project>
```



Pull & Sync changes from GitHub into VS Code

- **Pull:** download new commits and merge them into your local branch.
- **Fetch:** check for new commits without merging.



3.0 Cloning and pulling repositories

3.0.0 Cloning repositories

When you clone a repository, you copy the repository from GitHub.com to your local machine. You can clone a repository from GitHub.com to your local

computer to make it easier to fix merge conflicts, add or remove files, and push larger commits.

3.0.1 Pulling a Repository

What is pulling?

Pulling means **fetching the latest changes** (commits, updates, new files) from the remote GitHub repository and merging them into your **local repository**.

Where does it happen?

You perform git pull inside your **local project folder** (the one you cloned earlier).

Command:

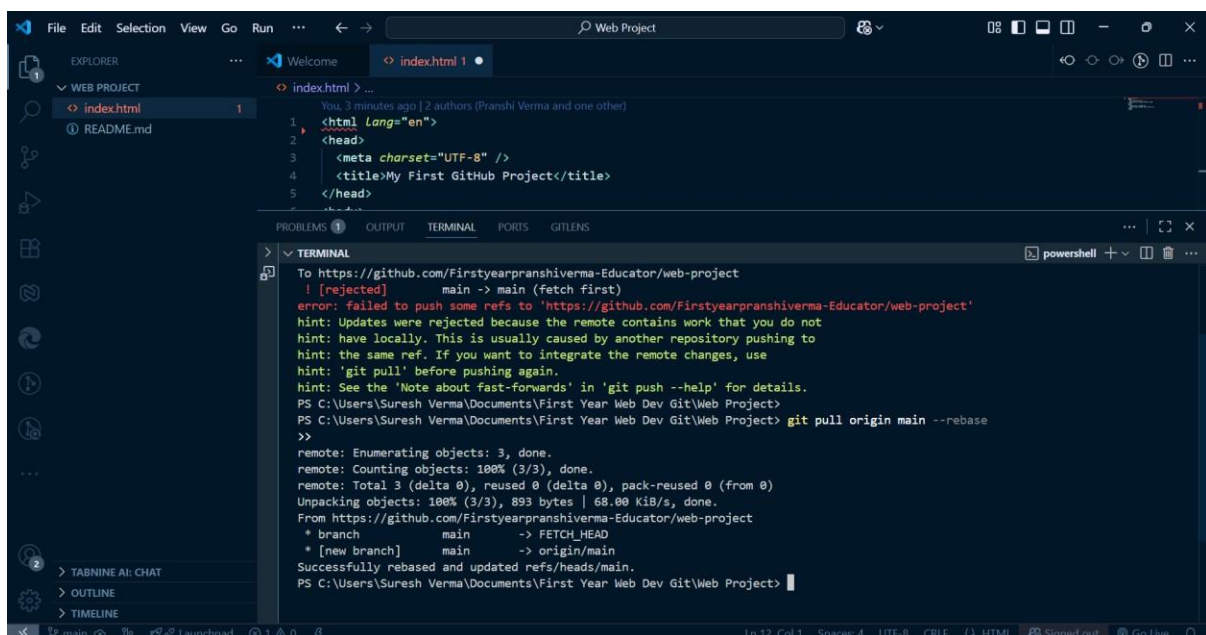
- `git pull origin main`
- `origin` refers to your GitHub repository.
- `main` is the branch name (sometimes `master` or another branch).

When do we use pull?

- If your team members (or you from another system) made changes on GitHub, you run `git pull` to **sync your local project** with the latest version.
- This avoids conflicts and keeps your work up-to-date.

In simple terms:

- **Clone** = "Download the whole project to my computer."
- **Pull** = "Update my local project with the latest changes from GitHub."

A screenshot of the Visual Studio Code (VS Code) interface. The Explorer pane on the left shows a project named 'WEB PROJECT' with files 'index.html' and 'README.md'. The main editor area displays the content of 'index.html', which is an HTML document with a title 'My First GitHub Project'. The TERMINAL pane at the bottom shows the output of a `git pull` command. The output indicates that the local branch 'main' was updated from the remote 'origin/main' by fetching and rebasing. The terminal text includes: 'To https://github.com/Firstyearpranshivarma-Educator/web-project', 'error: failed to push some refs to \'https://github.com/Firstyearpranshivarma-Educator/web-project\'', 'hint: Updates were rejected because the remote contains work that you do not have locally. This is usually caused by another repository pushing to the same ref. If you want to integrate the remote changes, use \'git pull\' before pushing again.', 'hint: See the \'Note about fast-forwards\' in \'git push --help\' for details.', 'PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git pull origin main --rebase', and 'Successfully rebased and updated refs/heads/main.' The status bar at the bottom shows 'main', 'Ln 12, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', and 'HTML'.

So Pull remote first, then push.

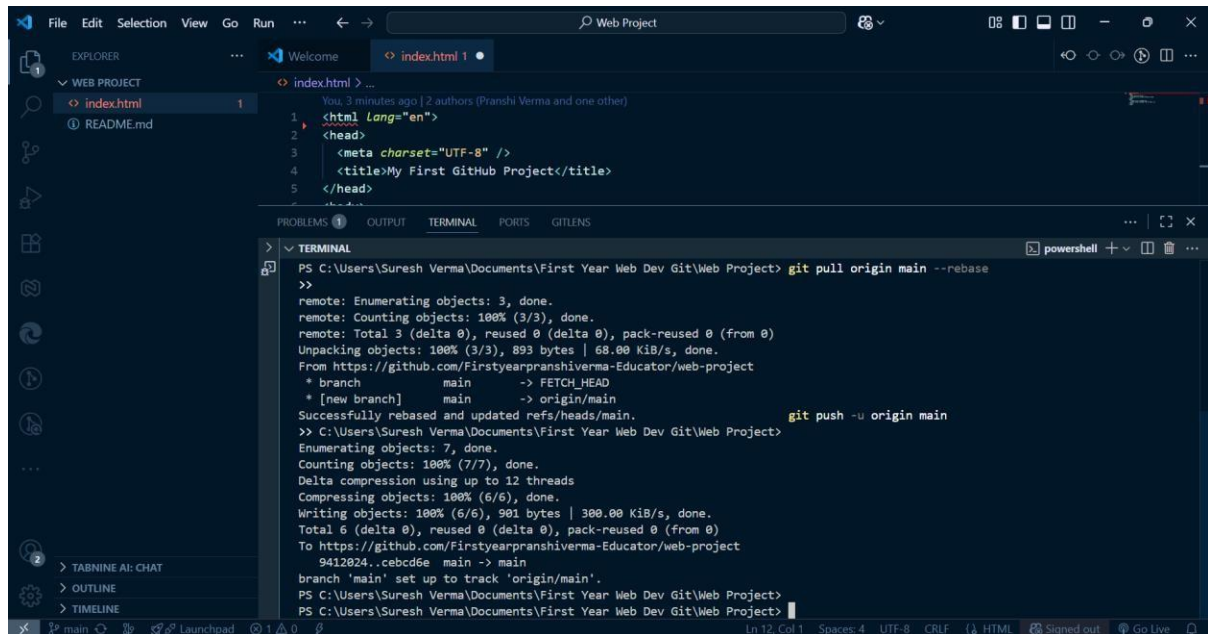
This way you combine your local and GitHub changes. Run:

- If there are no conflicts, your local commit will be placed “on top” of the remote commit.
- Then push:

Run the command:

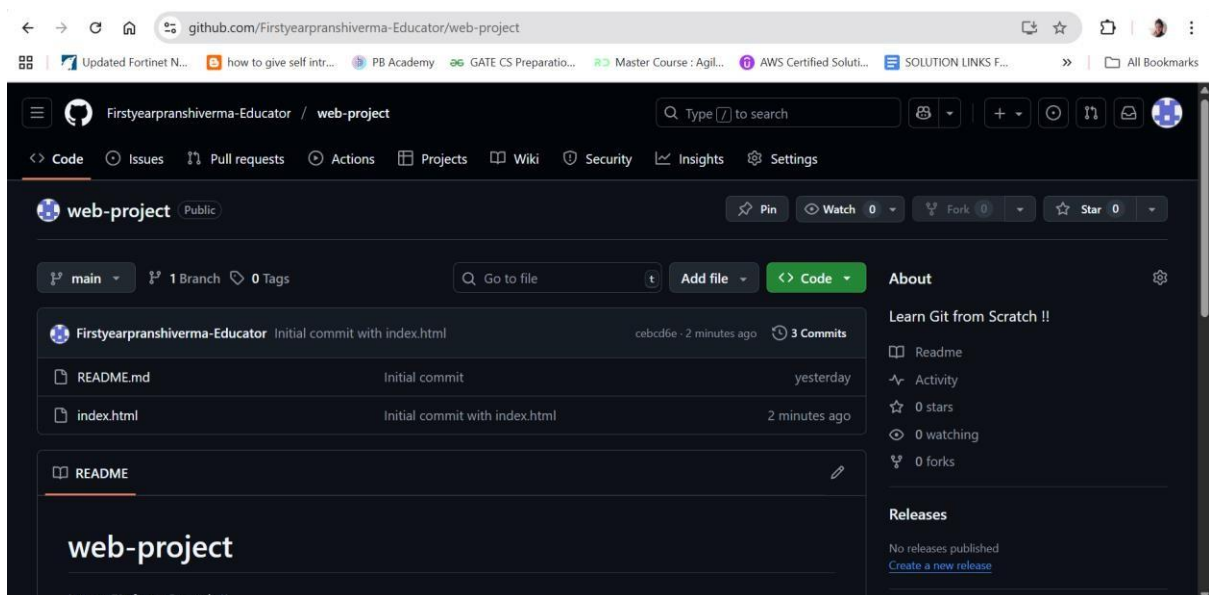
`git push -u origin main`

This means Upload my local main branch to the remote repository named origin, and set up a tracking relationship so future pushes/pulls are easier.



```
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git pull origin main --rebase
>>
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 893 bytes | 68.00 KiB/s, done.
From https://github.com/Firstyearpranshivarma-Educator/web-project
* branch      main      -> FETCH_HEAD
* [new branch] main      -> origin/main
Successfully rebased and updated refs/heads/main.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 901 bytes | 300.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshivarma-Educator/web-project
9412024..cebcd6e  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project>
```

To verify this, please refresh the GitHub repository and see the changes in files available in repository.



3.1 GitHub repository structure and README.md

3.1.0 GitHub Repository Structure

When you create or work with a GitHub repository, you will notice some important files and folders. Each has a specific purpose and makes your project more professional and maintainable:

3.1.1 README.md

What is README.md?

- A **README.md** file is written in **Markdown format** (file extension: .md).
- It provides an **introduction and overview of your project** so that visitors can quickly understand:
 - **What the project is about**
 - **How to install and run it**
 - **Features and usage**
 - **Contribution guidelines**
 - **Credits and contributors**

Example of a README.md file:

1. Project Title & Description

My Web Project

A simple web project to demonstrate Git & GitHub basics.

- The # symbol in Markdown means **Heading 1**.
 - Provide a **clear project name** and a **one-line description**.
-

2. Installation Instructions

Installation

Clone the repo:

<https://github.com/Firstyearpranshiverma-Educator/web-project.git>

Open the folder and run:
npm install (if Node.js project)

- Always guide users **step by step** on how to set up your project.
-

3. Usage Instructions

Usage

Open `index.html` in your browser.

- Show how to **run the project** after installation.
-

4. Contribution Guidelines

Contribution

Anyone can fork this repository, create a new branch, make improvements, and open a Pull Request for review.

- **Fork** → Copy someone else's repository into your own GitHub account.
 - You can **make changes freely** in your fork without affecting the original project.
 - Later, you can **create a Pull Request (PR)** to merge your changes into the original repository.
(We will discuss Fork and Pull Request in detail in upcoming lectures.)
-

5. Screenshots or Examples

Screenshots

We can add images like images/demo.png or any other demo images here.

- **Visuals** make your README attractive and easy to understand.
-

6. Credits & Contributors

##Credits & Contributors

- **Author / Maintainer: ** [Your Name](https://github.com/your-username)
- **Contributors: **

- [Contributor 1] (<https://github.com/contributor1>) – Feature XYZ
- [Contributor 2] (<https://github.com/contributor2>) – Bug fixes & testing

Special thanks to:

- Open-source libraries and frameworks used in this project
- Community members who gave feedback and suggestions

Explanation:

- **Author / Maintainer** → The person who created or maintains the repo.
- **Contributor's** → People who added code, fixed bugs, improved docs, etc.
- **Special Thanks** → Credit libraries, tutorials, or external help. *(GitHub also auto-generates a contributors list based on commits, but mentioning key contributors manually is a good practice.)*

Best Practice:

- Always write a **clear and meaningful README.md** because it's the **first impression** of your project on GitHub.
 - Keep it **organized, simple, and helpful** for anyone who visits your repository.
-

2. .gitignore

- This file tells Git which files or folders **should not be tracked** or uploaded to GitHub.
- Example use cases:
 - Ignoring dependencies like `node_modules/` in Node.js projects
 - Ignoring log files (`.log`)
 - Ignoring environment configuration files (`.env`) for security
- Example .gitignore content:

```
node_modules/
```

```
.env
```

```
*.log
```

3. LICENSE

- This file specifies the **license type** under which your project is distributed.

- It defines what other developers can and cannot do with your code (e.g., MIT, Apache 2.0, GPL).
- Example:
 - MIT License → Free to use, copy, modify with attribution
 - GPL License → Derivative projects must also be open-source

Including a **LICENSE file** makes your project legally safe and encourages contributions.

4. src/ (Source Code Folder)

- The src/ directory usually contains the **main source code** of your project.
- Example structure for a web project:

src/

├── index.html

├── style.css

├── script.js

└── images/

- Keeping source files inside src/ makes your repository **organized and professional**.

Key Takeaway

- **README.md** → First impression (must be meaningful and well-written).
 - **.gitignore** → Keeps unnecessary files out of Git.
 - **LICENSE** → Defines usage rights of your project.
 - **src/** → Main code lives here (clean and organized).
-

3.2 Basic collaboration terms: fork, pull request, branch (intro)

3.2.0 Basic Collaboration Terms: Fork

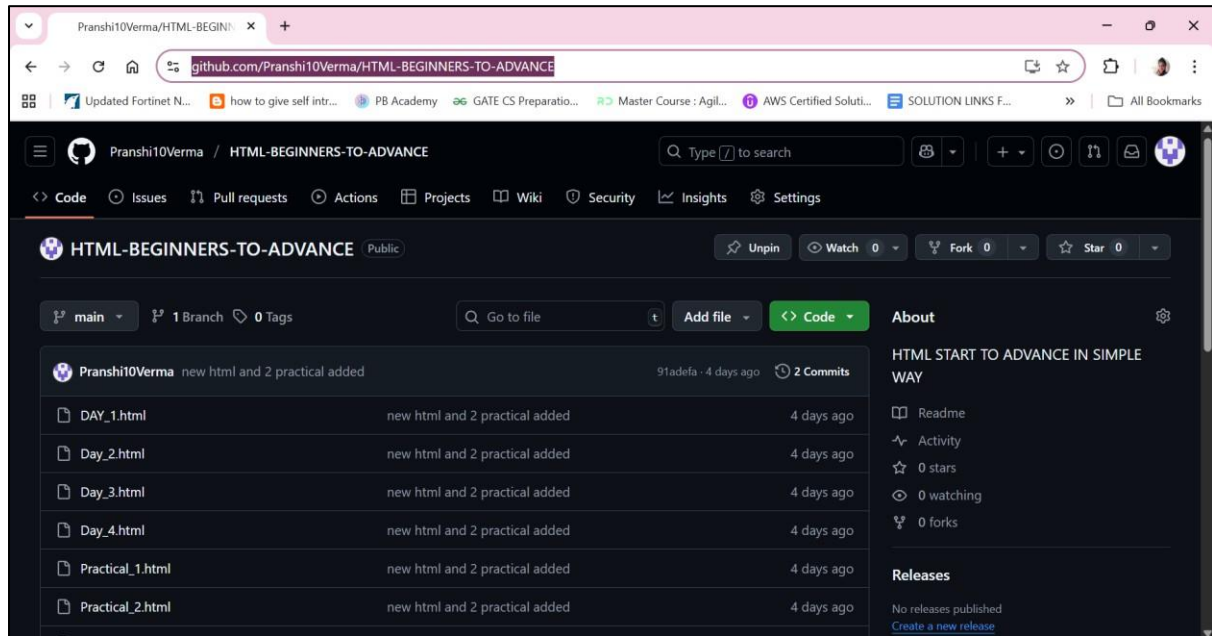
- A **Fork** is a personal copy of someone else's repository on your GitHub account.
- It allows you to experiment with changes **without affecting the original repository**.

- Commonly used in **open-source projects** where you don't have write access to the original repo.

Example:

Suppose there is a public repo:

🔗 <https://github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE>



Step 1: Open the repository on GitHub

- Go to the repo link in your browser.
- At the top-right of the page, you'll see a **Fork** button.
- This button is used to create your own copy of the repo inside your GitHub account.

🔗 Think of Fork as “Save a copy to my account.”

Step 2: Click on Fork

- Click the **Fork** button.
- GitHub will ask where you want to fork the repository (if you belong to multiple organizations, you can select your personal account).
- Optionally, you can rename the repository during the fork.
- Then click **Create Fork**.

github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE/fork

Create a new fork

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *
Firstyearpranshivarma-Educator

Repository name *
Fork HTML-BEGINNERS-TO-ADVANCE

Your new repository will be created as Fork-HTML-BEGINNERS-TO-ADVANCE.
The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)
HTML START TO ADVANCE IN SIMPLE WAY

☒ Copy the main branch only
Contribute back to Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE by adding your own branch. [Learn more.](#)

i You are creating a fork in your personal account.

Create fork

You can click **Fork** (on GitHub UI) → it creates a copy in your account:

github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE/fork

Create a new fork

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *
Firstyearpranshivarma-Educator

Repository name *
Fork HTML-BEGINNERS-TO-ADVANCE

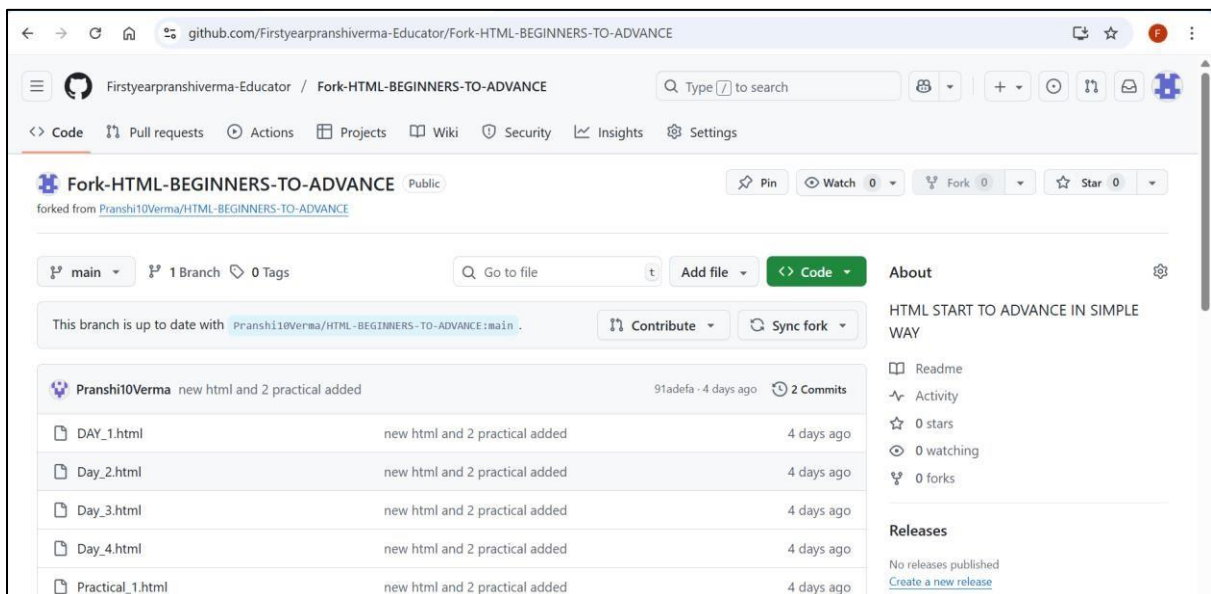
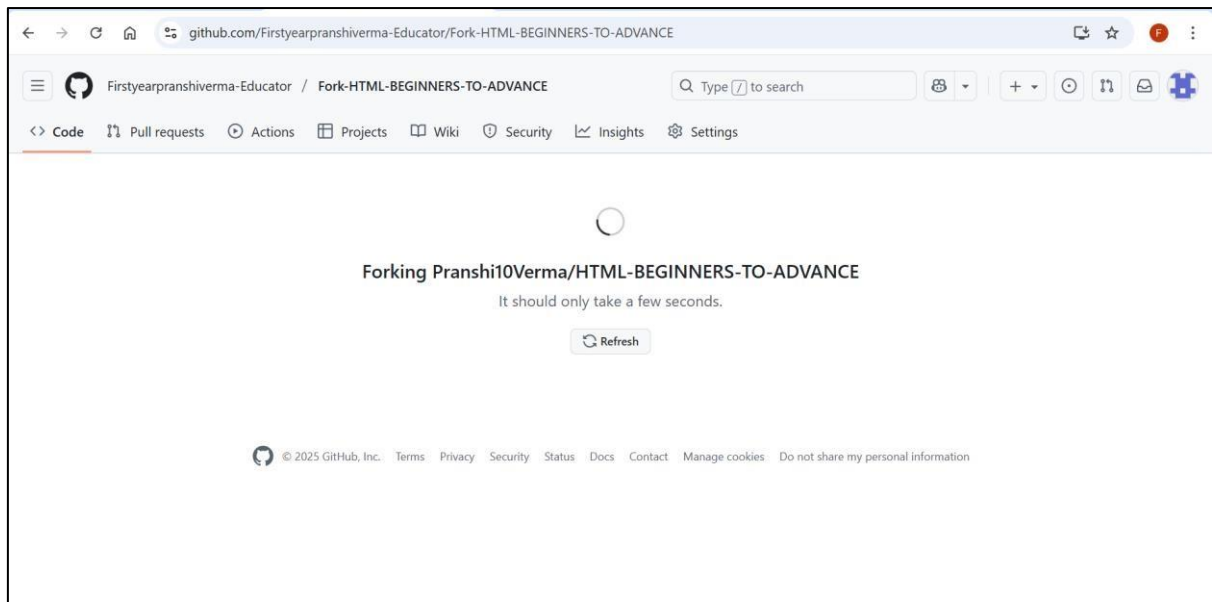
Your new repository will be created as Fork-HTML-BEGINNERS-TO-ADVANCE.
The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)
HTML START TO ADVANCE IN SIMPLE WAY

☒ Copy the main branch only
Contribute back to Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE by adding your own branch. [Learn more.](#)

i You are creating a fork in your personal account.



 <https://github.com/Firstyearpranshiverma-Educator/Fork-HTML-BEGINNERS-TO-ADVANCE>

Now you can make changes safely in your copy.

3.2.1 Basic Collaboration Terms: Pull Request

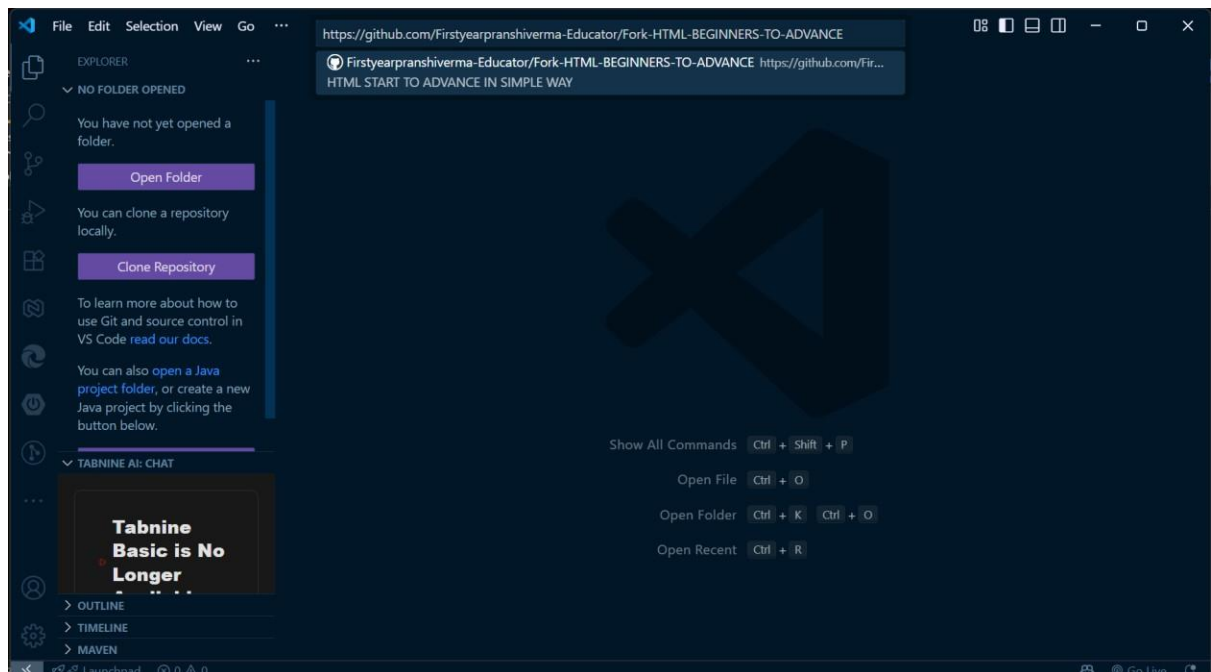
What is Pull Request (PR)?

A **Pull Request** is a request to merge your changes from your fork (or branch) back into the original repository.

- It's the main way developers contribute to open-source projects.

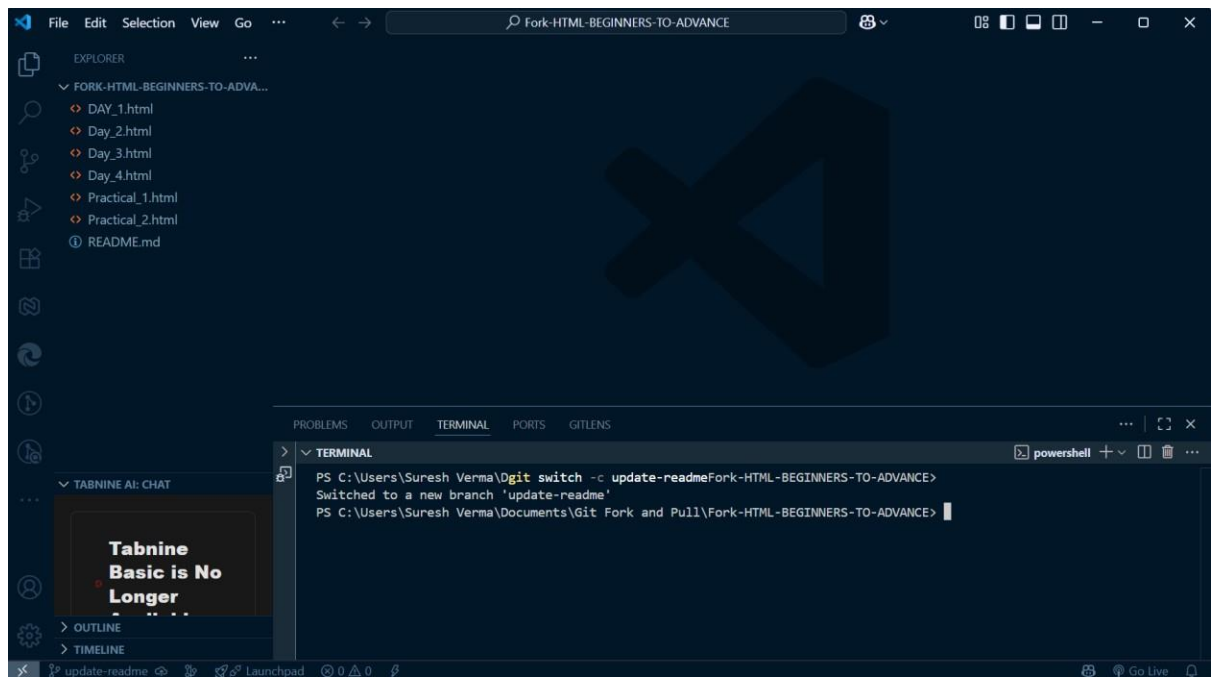
Clone Your Fork Locally

In VS Code:



Create a New Branch for Your Change

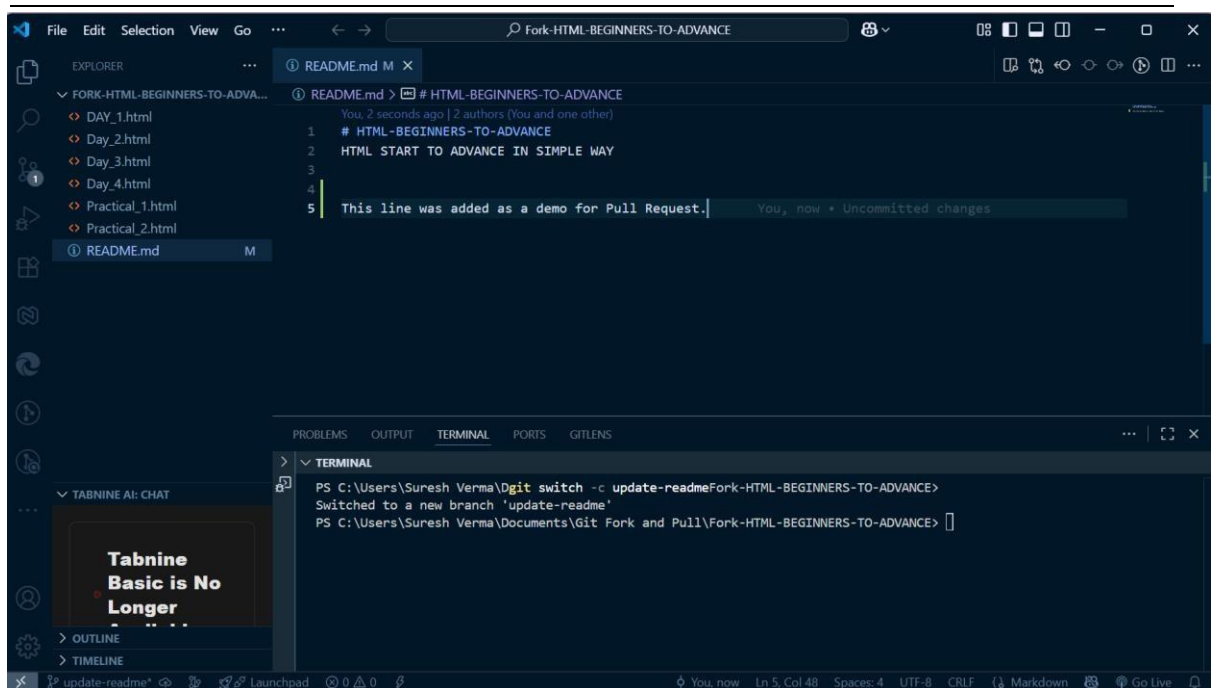
`git switch -c update-readme`



Make a Change

Open README.md and add a new line at the bottom:

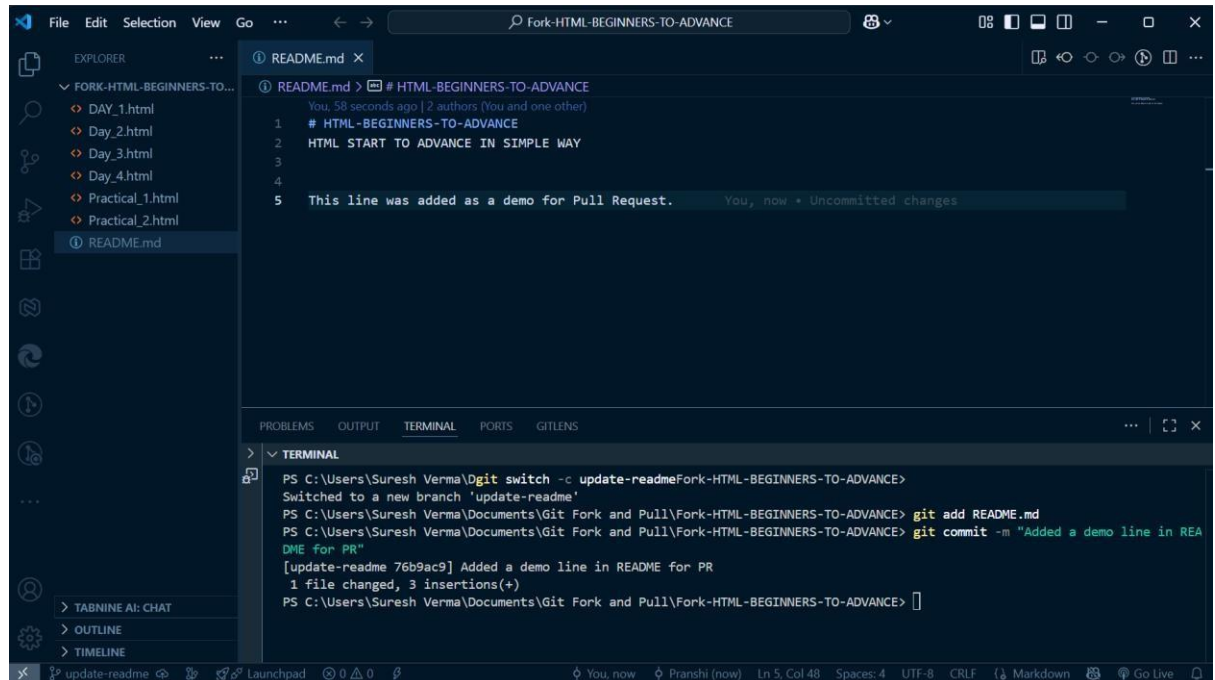
This line was added as a demo for Pull Request.



Commit the Change

`git add README.md`

`git commit -m "Added a demo line in README for PR"`

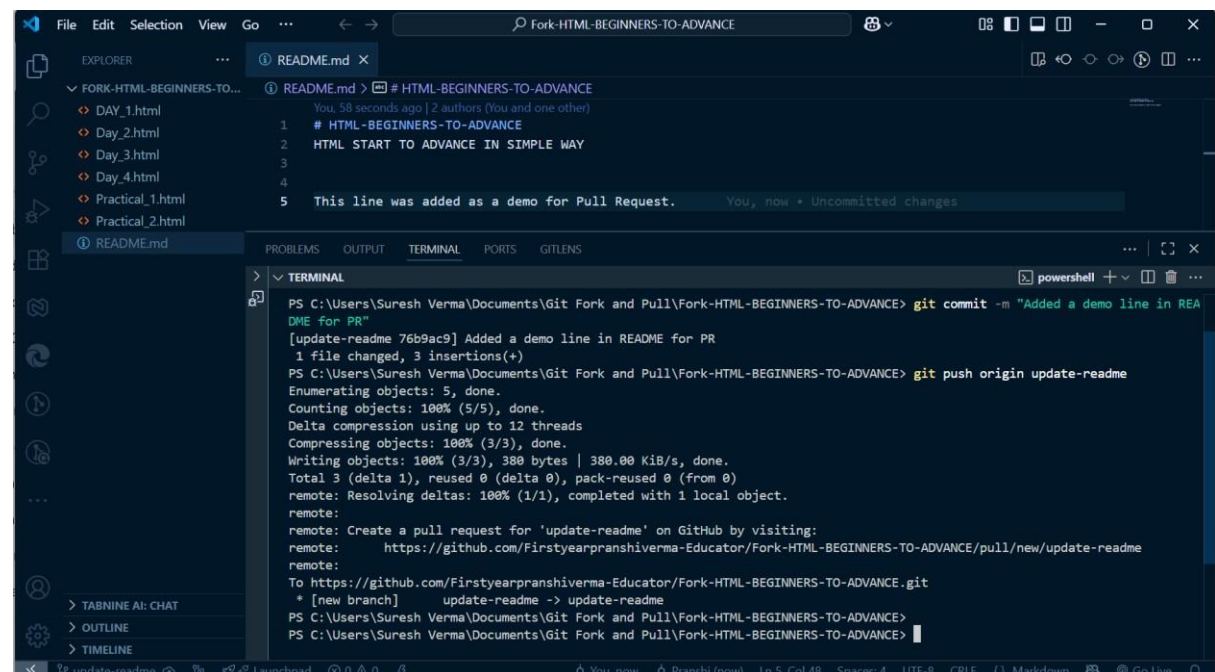


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'FORK-HTML-BEGINNERS-TO-ADVANCE' with files 'DAY_1.html', 'Day_2.html', 'Day_3.html', 'Day_4.html', 'Practical_1.html', 'Practical_2.html', and 'README.md'. The README.md file is open in the editor, showing its content. The terminal pane at the bottom shows the following commands and output:

```
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git add README.md
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git commit -m "Added a demo line in README for PR"
[update-readme 76b9ac9] Added a demo line in README for PR
1 file changed, 3 insertions(+)
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE>
```

Push the Branch to Your Fork

`git push origin update-readme`

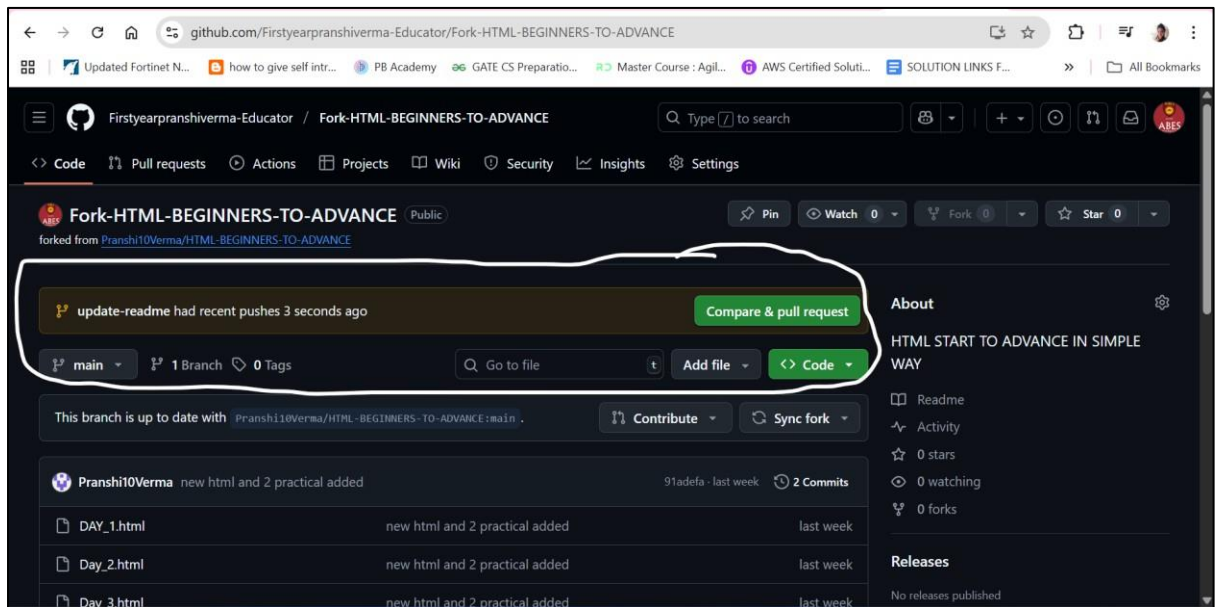


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows the same project as the previous screenshot. The terminal pane at the bottom shows the following commands and output:

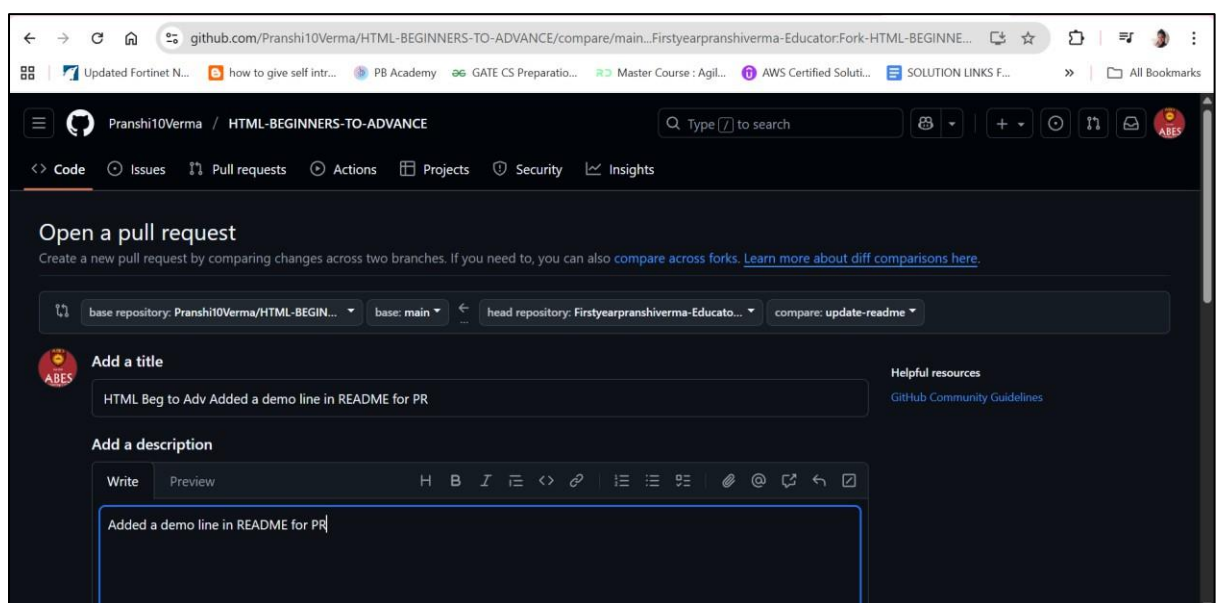
```
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE> git push origin update-readme
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 380 bytes | 380.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'update-readme' on GitHub by visiting:
remote:   https://github.com/Firstyearpranshiverma-Educator/Fork-HTML-BEGINNERS-TO-ADVANCE/pull/new/update-readme
remote:
To https://github.com/Firstyearpranshiverma-Educator/Fork-HTML-BEGINNERS-TO-ADVANCE.git
 * [new branch] update-readme -> update-readme
PS C:\Users\Suresh Verma\Documents\Git Fork and Pull\Fork-HTML-BEGINNERS-TO-ADVANCE>
```

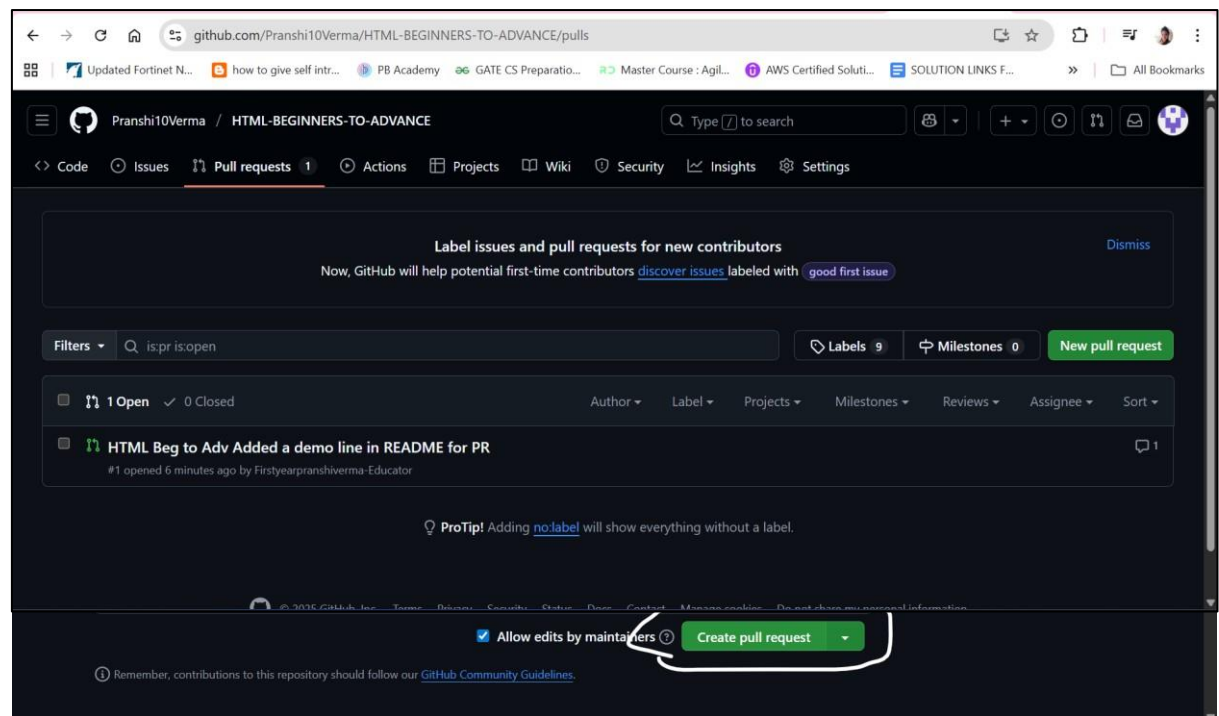
Open a Pull Request

- Go to your fork on GitHub.
- You'll see a message: *"Compare & pull request"*.
- Click it.

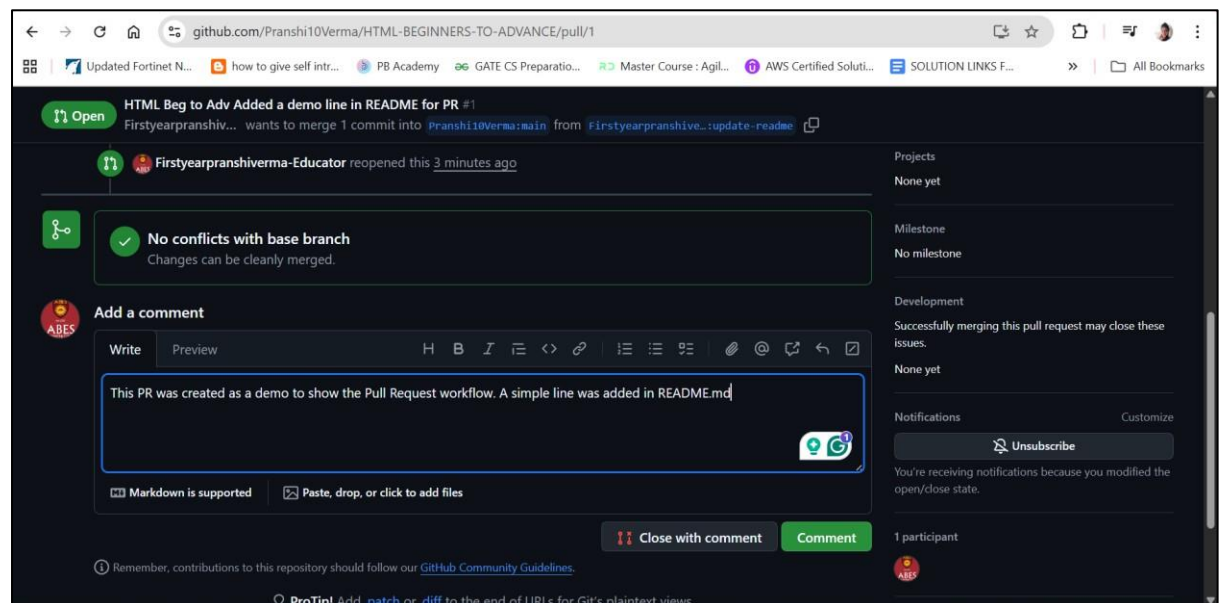


1. GitHub shows a comparison view:
 - **Base repository** → the original repo (octocat/Spoon-Knife).
 - **Head repository** → your fork + branch (your-username/update-readme).
2. Add a title and description.

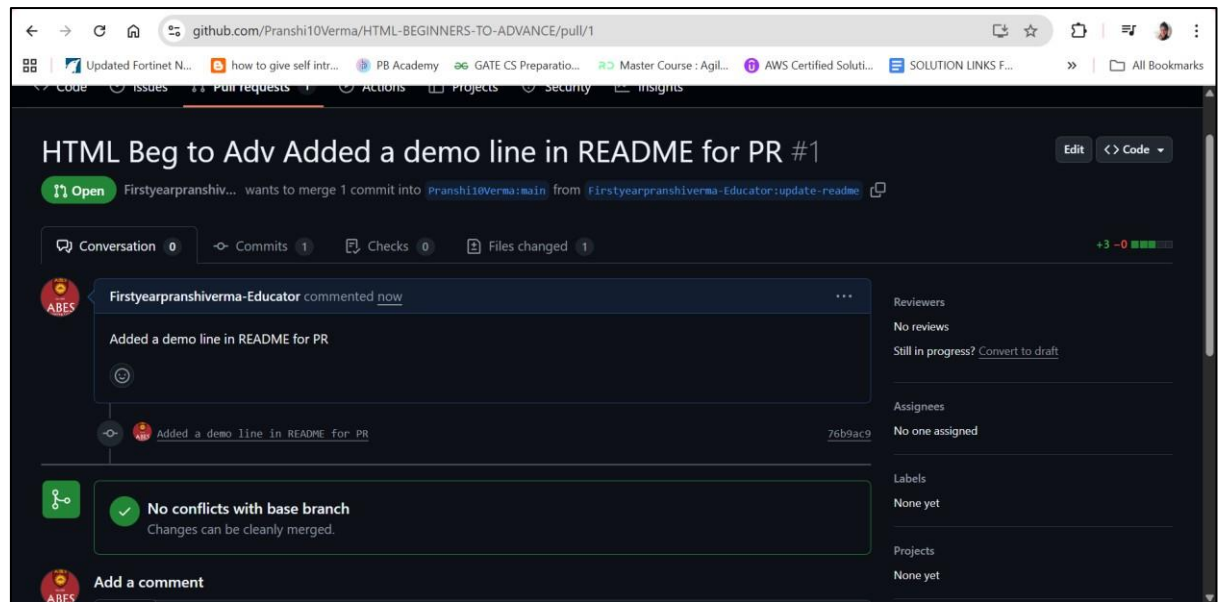




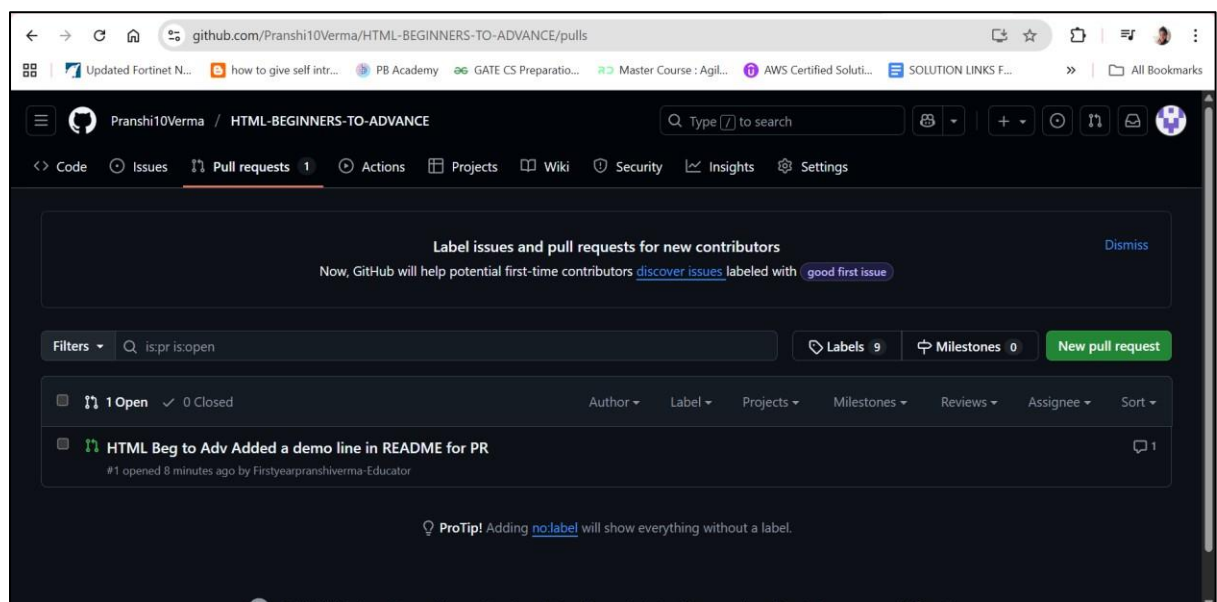
3. Click Create Pull Request.



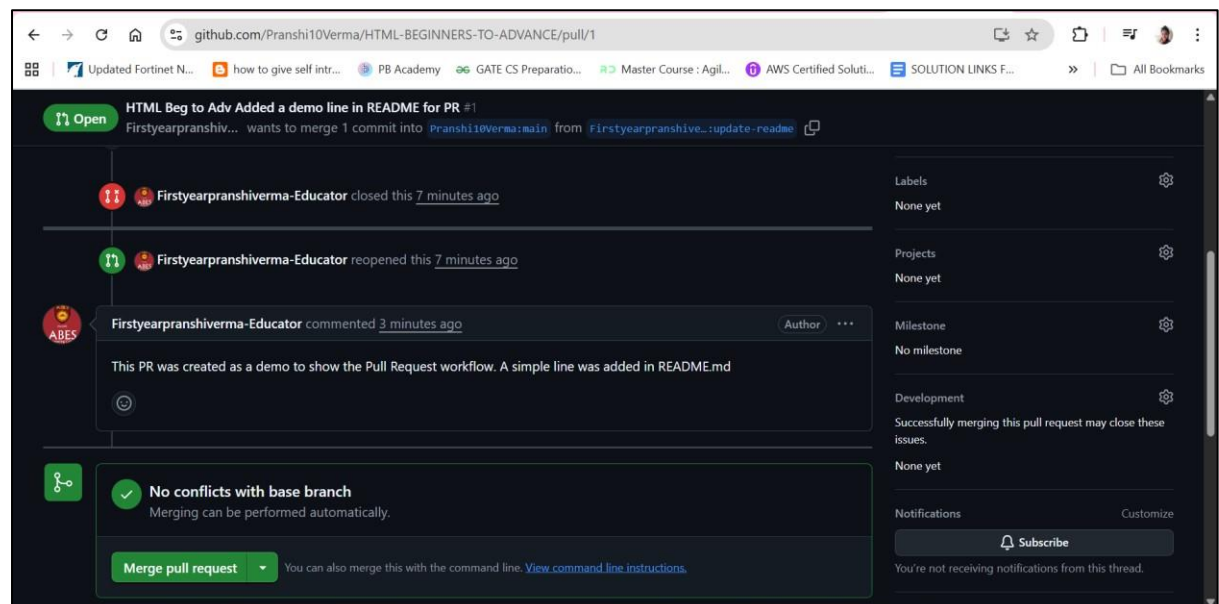
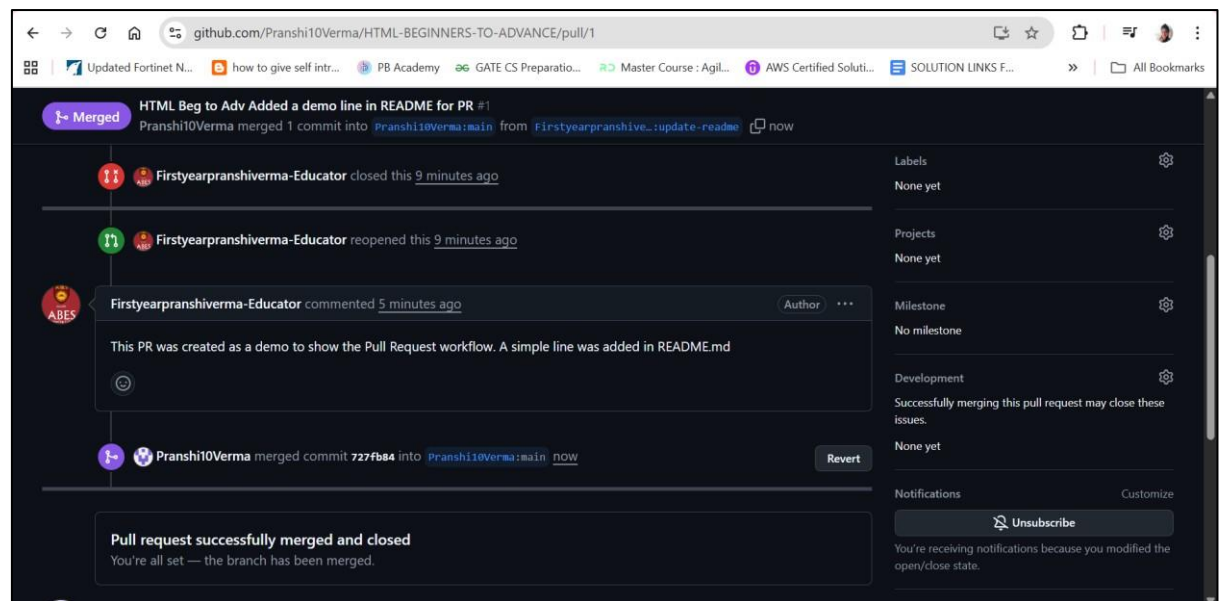
4. Add a comment for high readability. (not mandatory).



After that owner of the url <https://github.com/Pranshi10Verma/HTML-BEGINNERS-TO-ADVANCE> received a pull request



- you'll see a **Merge pull request** button.
- Click it → then click **Confirm merge**.



PR merged successfully.

Key Takeways:

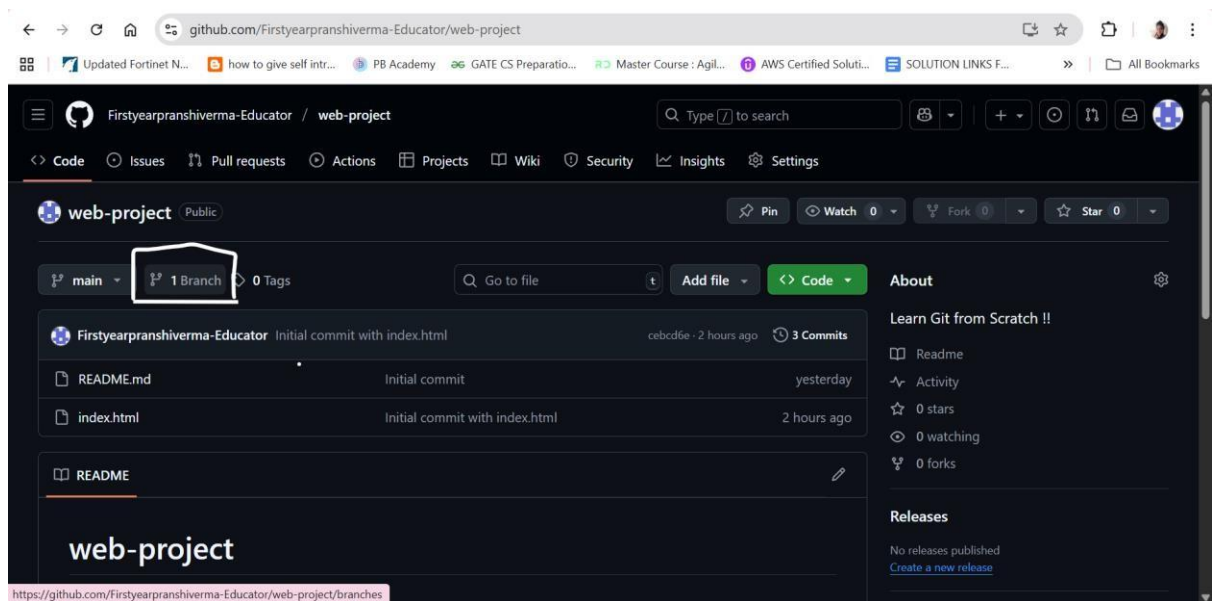
- Fork → Create personal copy of repo.
- Clone → Bring fork to your computer.
- Branch → Make safe changes.
- Commit & Push → Upload to fork.
- PR → Ask original repo owner to accept.
- Merge → Changes officially added.

3.3 Branching: git branch, git checkout, git switch, merging branches, handling merge conflicts

3.3.0 Branch

- A **Branch** is a parallel line of development inside a repository.

By default, every repo has a branch called **main** (or master).



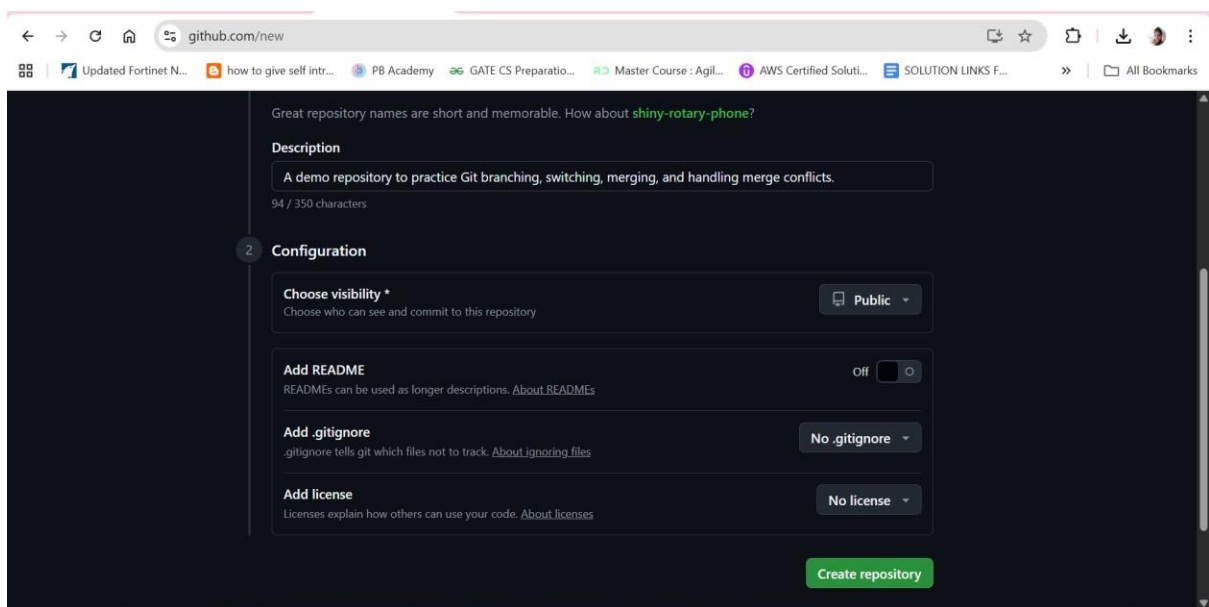
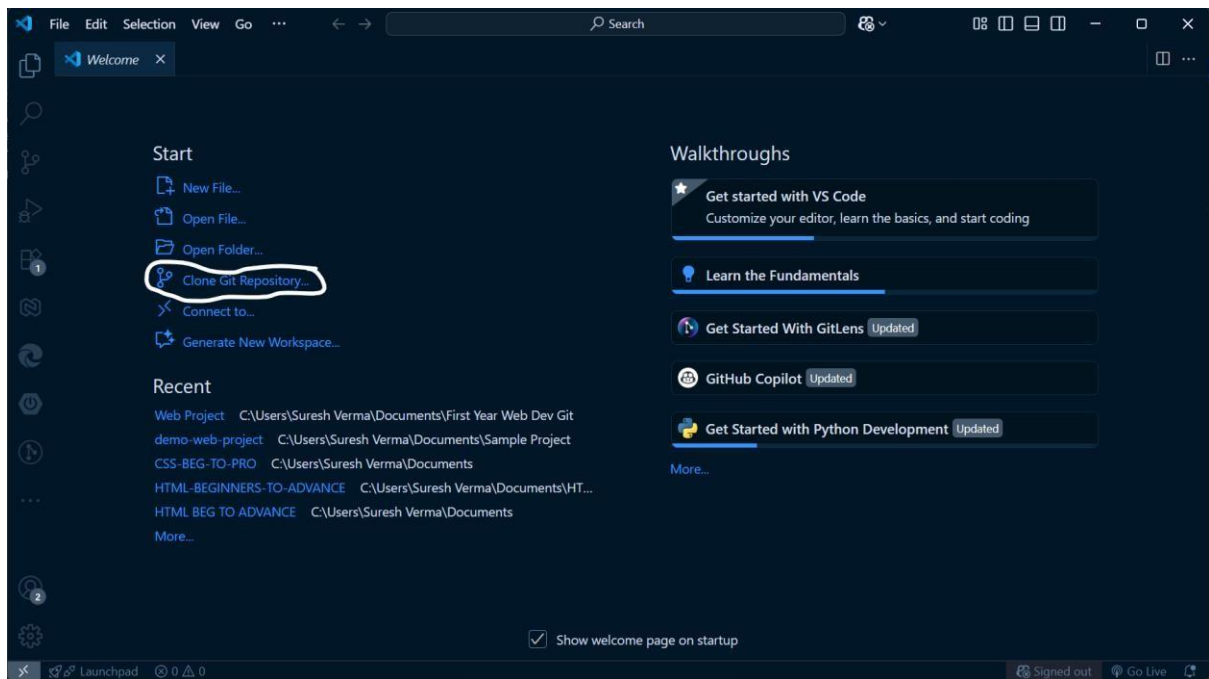
- Developers create new branches to work on features, bug fixes, or experiments **without disturbing the main branch**.

Let's understand how to create branch, switching between branches, making changes, and merging branches.

- You can create new branches to work on features or fixes without disturbing the main project.

Step 1: Create a Repository on GitHub

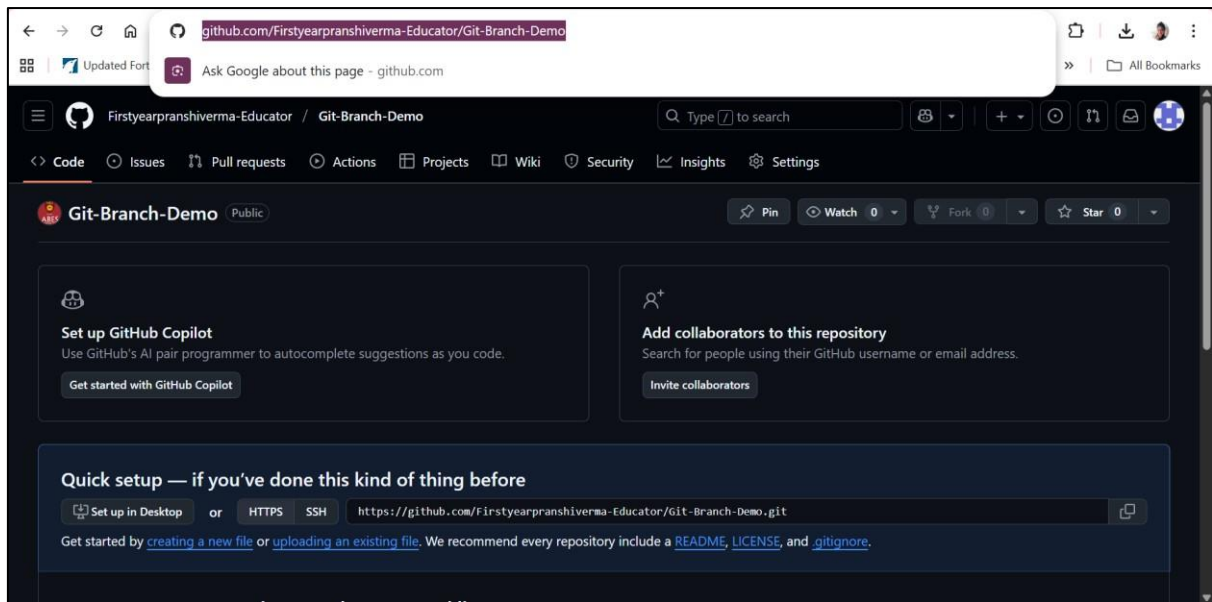
1. Log in to your GitHub account.
2. Click on **New Repository**.
3. Enter the repository name → **Git Branch Demo**.
4. Do **not** check the option *"Initialize this repository with a README"*.
5. Click **Create Repository**.



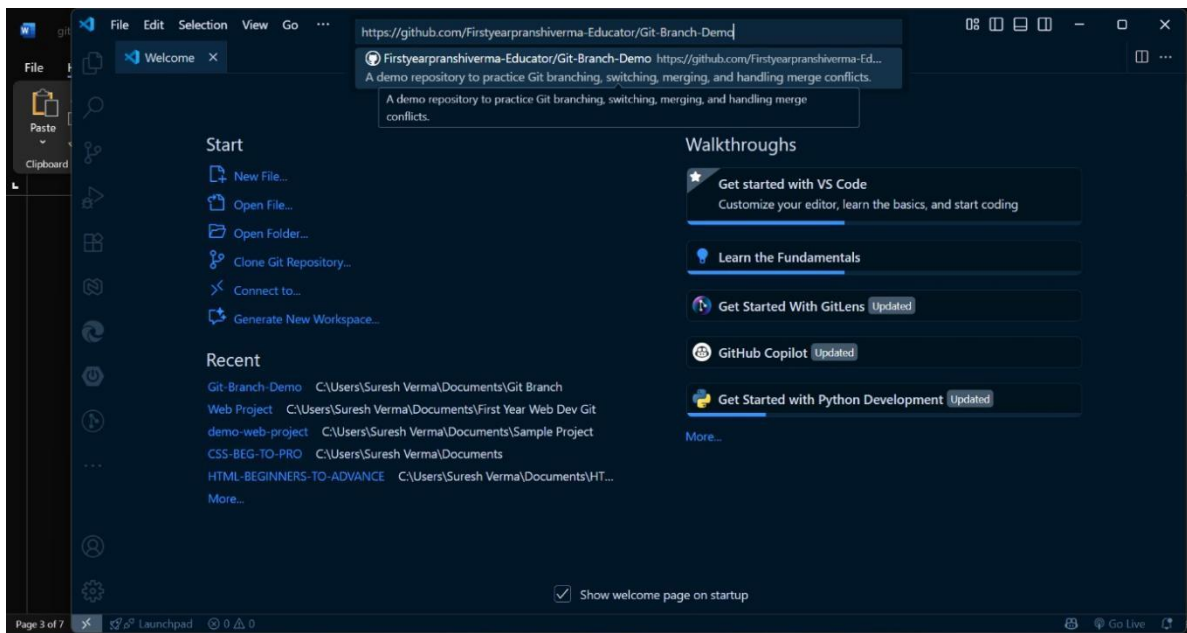
Step 2: Clone the Repository into VS Code

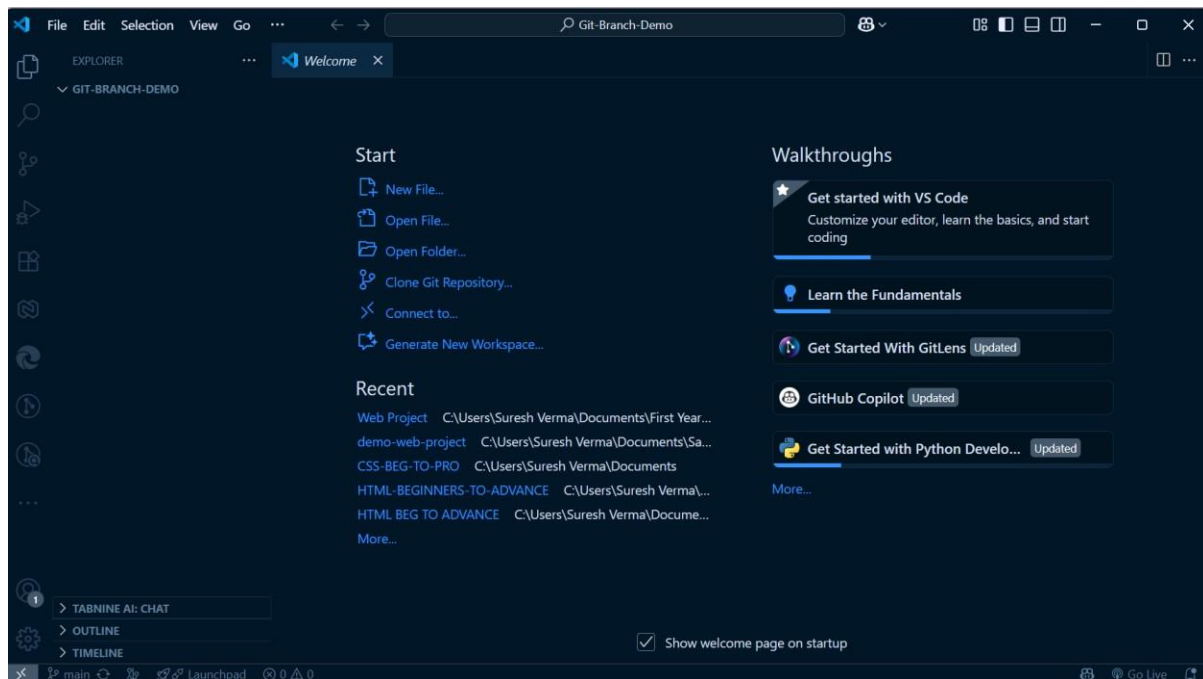
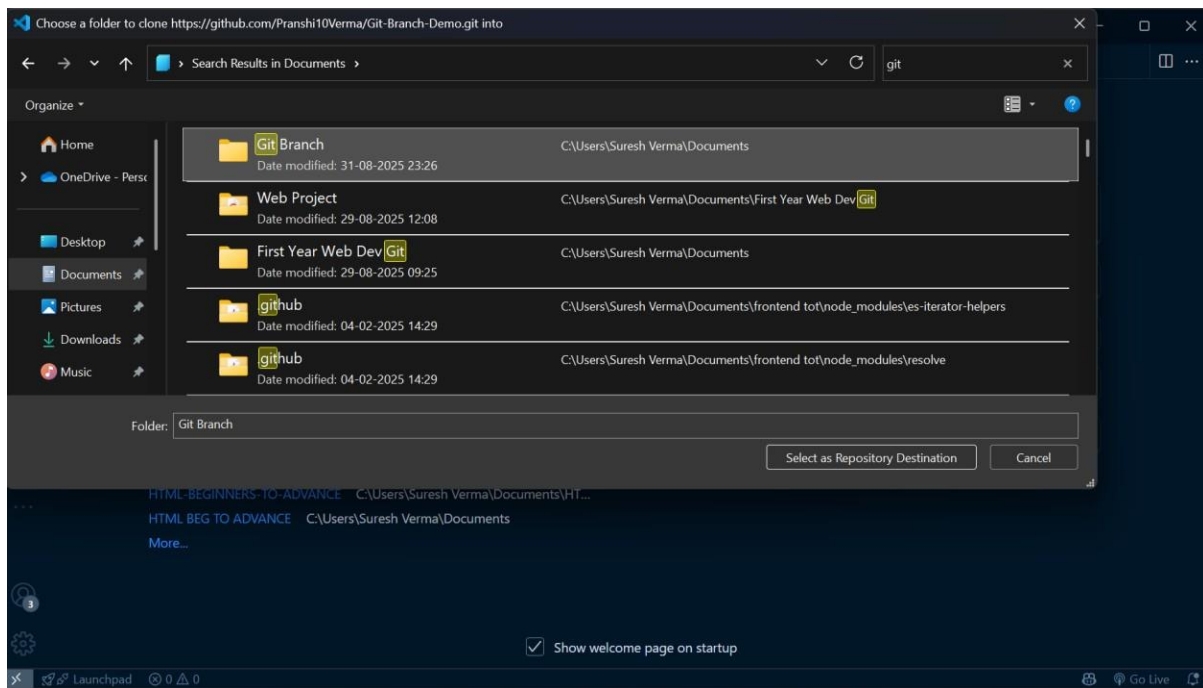
Open the VS Code and click on Clone Git Repository...

Then copy the url from github repo.



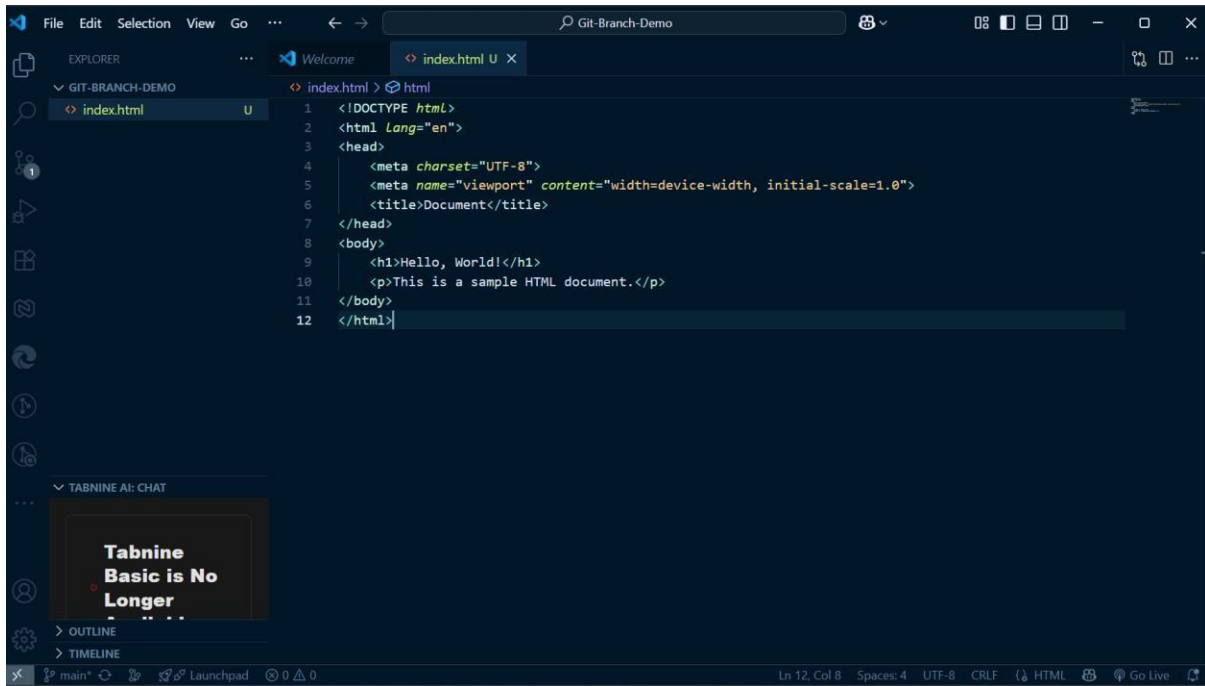
Then select the folder, where you want to clone it





Repository cloned successfully

Step 4: Create a new file named index.html and add the content:

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a folder named 'GIT-BRANCH-DEMO' with a file 'index.html' inside. The main editor area shows the content of 'index.html' with the following code:

```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11 </body>
12 </html>
```

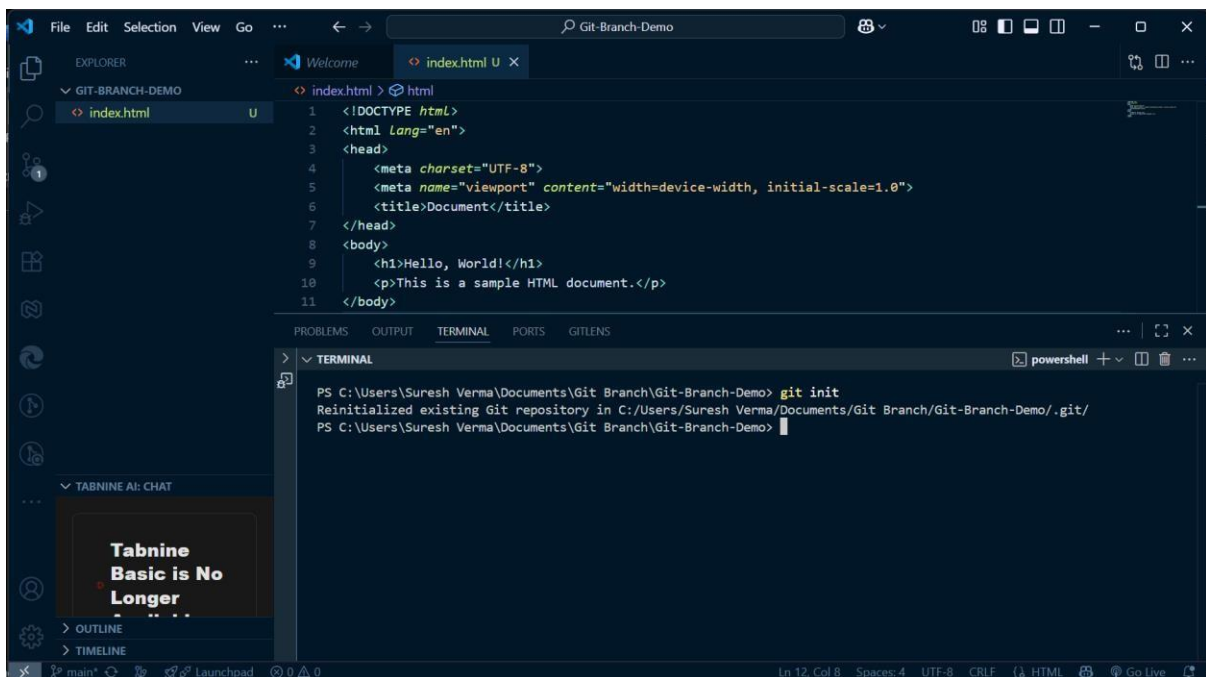
The status bar at the bottom indicates 'Ln 12, Col 8', 'Spaces: 4', 'UTF-8', 'CRLF', and 'HTML'.

Step 5: Initialize Git (if not already initialized)

In the VS Code terminal:

```
git init
```

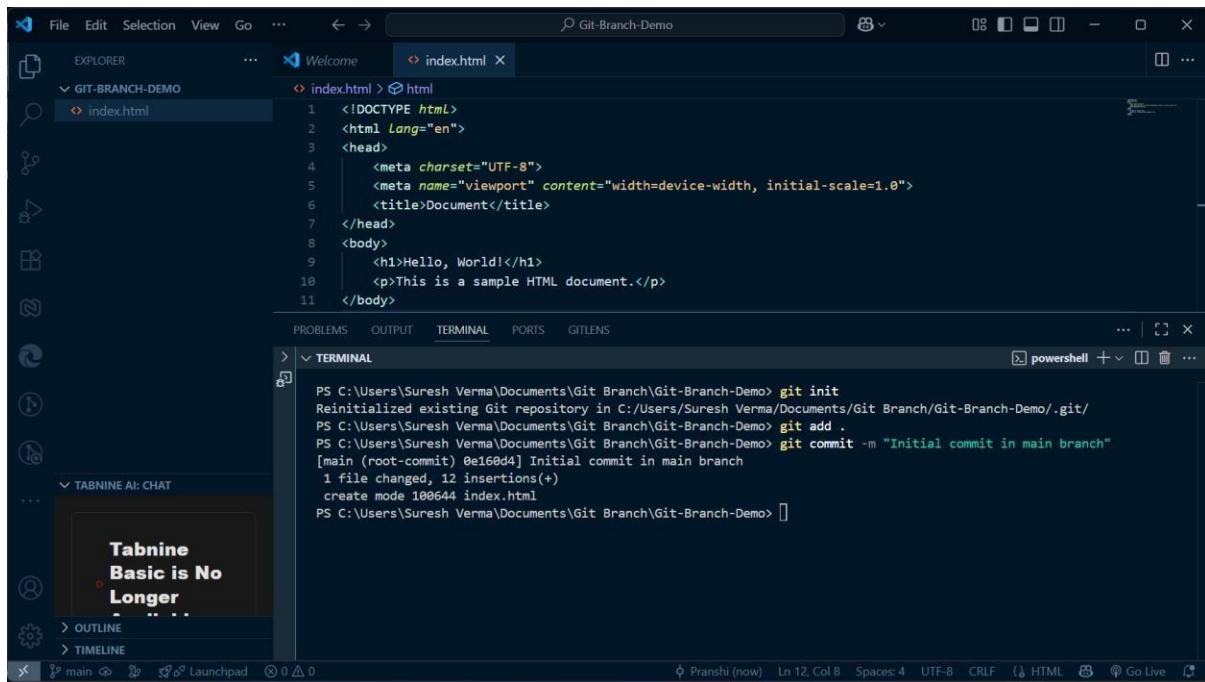
This command initializes the folder as a Git repository.

A screenshot of the Visual Studio Code editor interface, showing the terminal output of the 'git init' command. The terminal window is titled 'TERMINAL' and shows the following output:

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git init
Reinitialized existing Git repository in C:/Users/Suresh Verma/Documents/Git Branch/Git-Branch-Demo/.git/
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

The status bar at the bottom indicates 'Ln 12, Col 8', 'Spaces: 4', 'UTF-8', 'CRLF', and 'HTML'.

Step 6: Make the First Commit

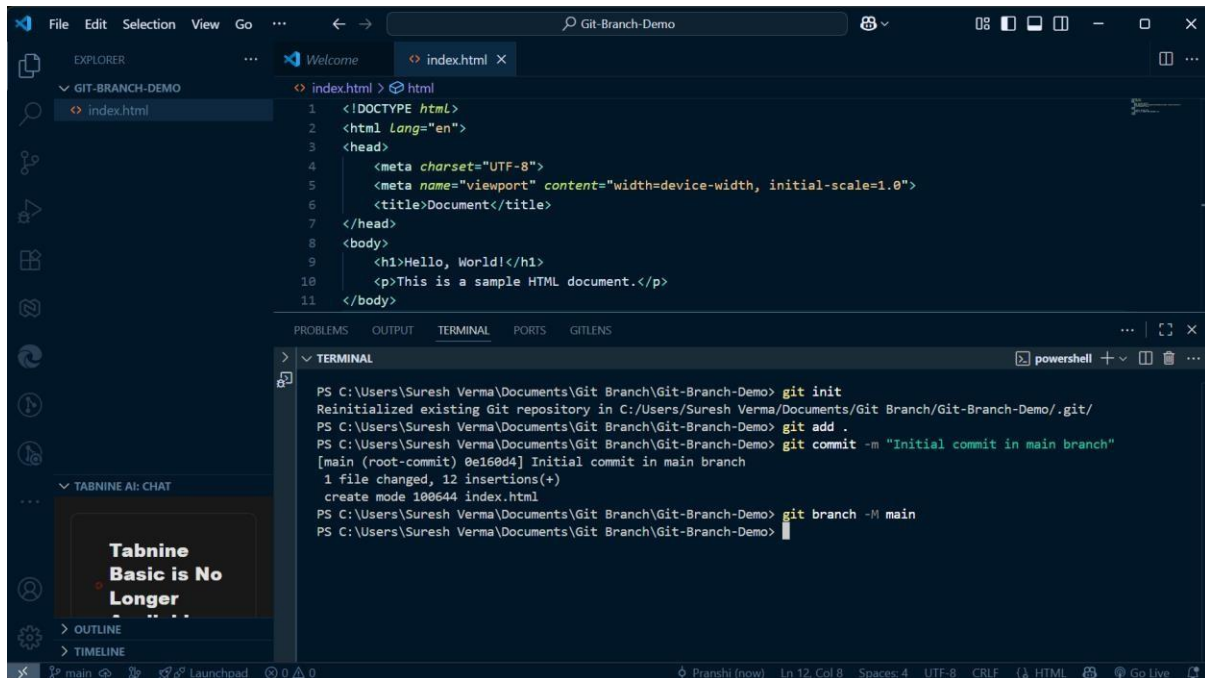


The screenshot shows the Visual Studio Code interface with a file explorer on the left showing a project named 'GIT-BRANCH-DEMO' containing 'index.html'. The editor displays the content of 'index.html', which is a basic HTML document. The terminal at the bottom shows the execution of the following commands:

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git init
Reinitialized existing Git repository in C:/Users/Suresh Verma/Documents/Git Branch/Git-Branch-Demo/.git/
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add .
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Initial commit in main branch"
[main (root-commit) 0e160d4] Initial commit in main branch
1 file changed, 12 insertions(+)
create mode 100644 index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Step 7: Push the Code to GitHub

git branch -M main



The screenshot shows the Visual Studio Code interface with the same file explorer and editor as in Step 6. The terminal at the bottom shows the execution of the following commands:

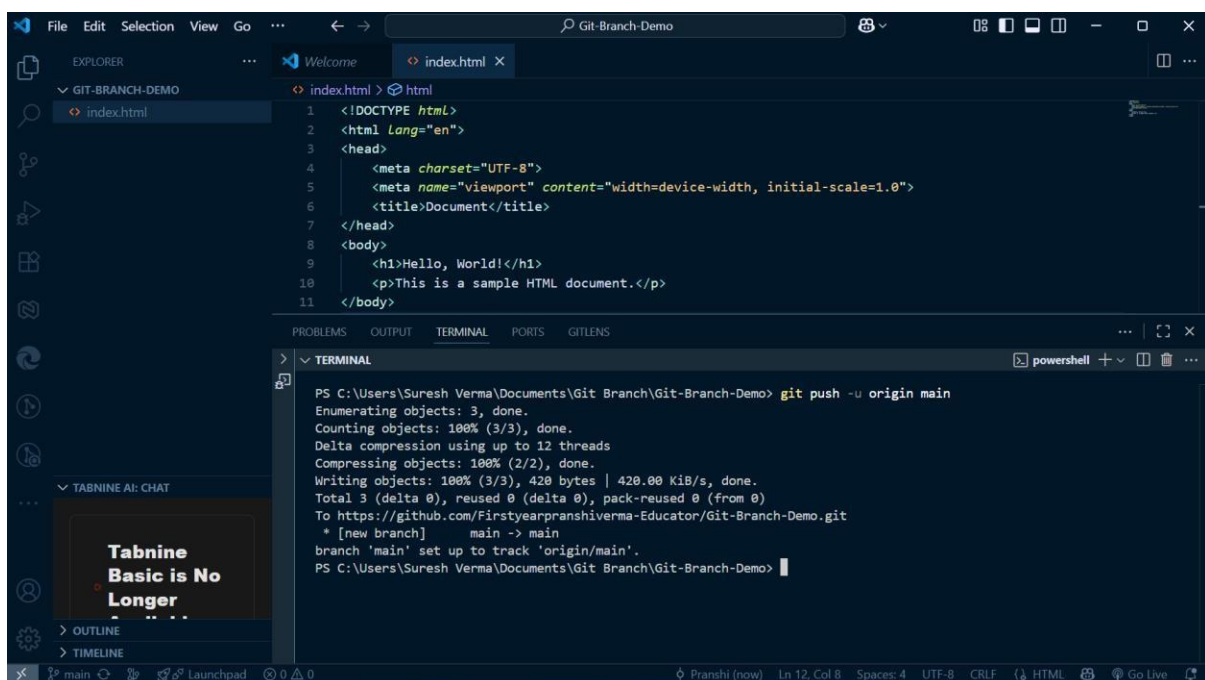
```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git init
Reinitialized existing Git repository in C:/Users/Suresh Verma/Documents/Git Branch/Git-Branch-Demo/.git/
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add .
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Initial commit in main branch"
[main (root-commit) 0e160d4] Initial commit in main branch
1 file changed, 12 insertions(+)
create mode 100644 index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch -M main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Meaning:

- `git branch` → command to manage branches.
- `-M` → option that means **move/rename branch (forcefully if needed)**.
- `main` → the new name for the current branch.

So this command **renames your current branch to main**, even if a branch with that name already exists (it will overwrite).

`git push -u origin main`

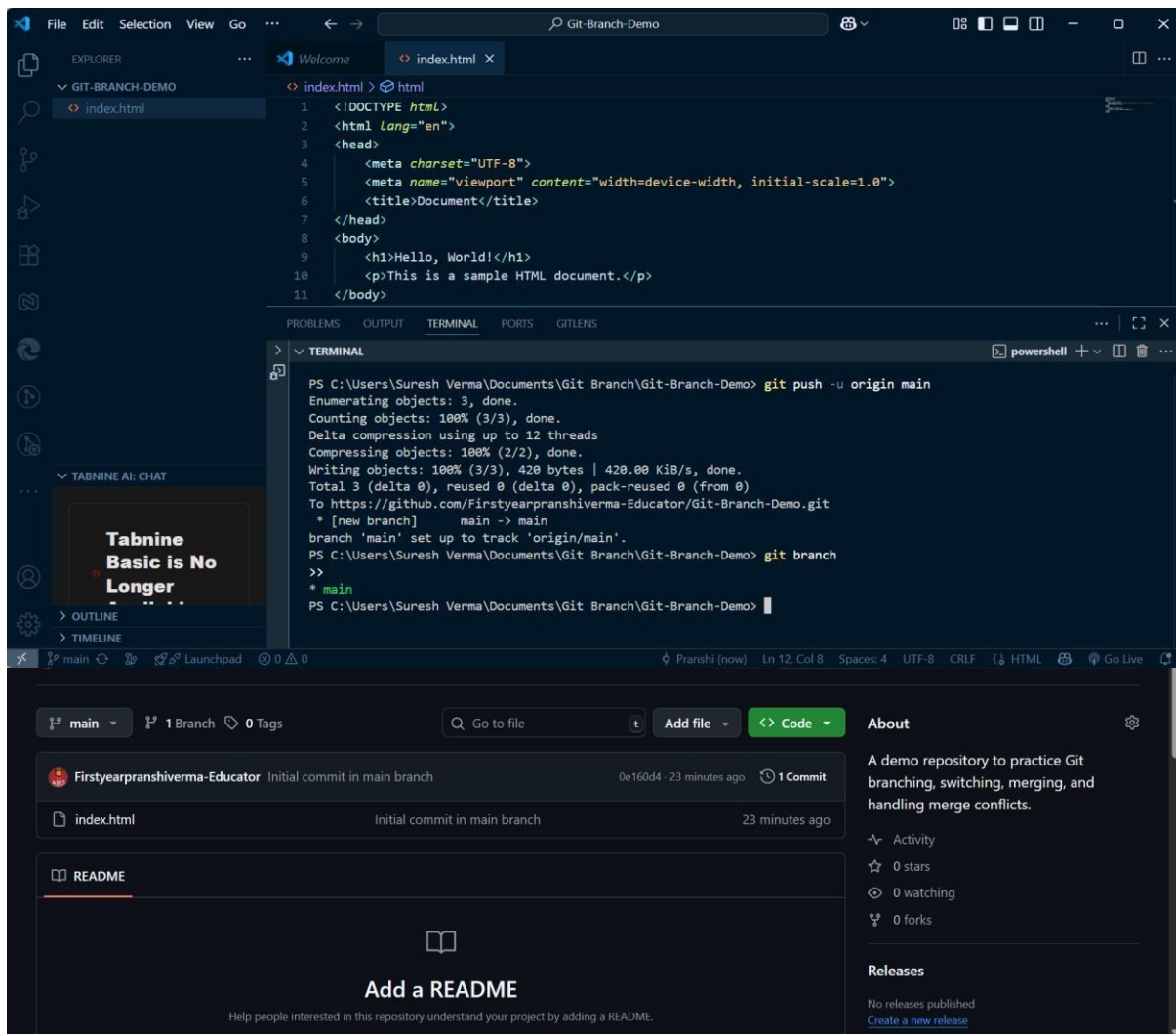


The screenshot shows a Visual Studio Code editor window titled 'Git-Branch-Demo'. The Explorer sidebar on the left shows a file named 'index.html'. The main editor area displays the content of 'index.html', which is an HTML document with a title 'Document' and a body containing 'Hello, World!' and 'This is a sample HTML document.' Below the editor, the TERMINAL panel is open, showing the output of the command `git push -u origin main` executed in a PowerShell terminal. The output indicates that the local branch 'main' is being pushed to the remote repository 'origin/main'.

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 420 bytes | 420.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshiverma-Educator/Git-Branch-Demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Meaning:

- **git push**
Uploads your local commits (from your computer) to a remote repository (like GitHub).
- **origin**
This is the short name (alias) for the remote repository you linked earlier using
`git remote add origin <URL>`
- By convention, the first/primary remote is always called **origin**.
- **main**
The branch you want to push. Here it means: *“Push the local main branch to the remote repository’s main branch.”*



- **-u (or --set-upstream)**
This sets a **tracking relationship** between your local main branch and the remote main branch.
After this first time, you don't need to write `git push origin main` again.
You can simply run:

`git push`

and Git will know where to push by default.

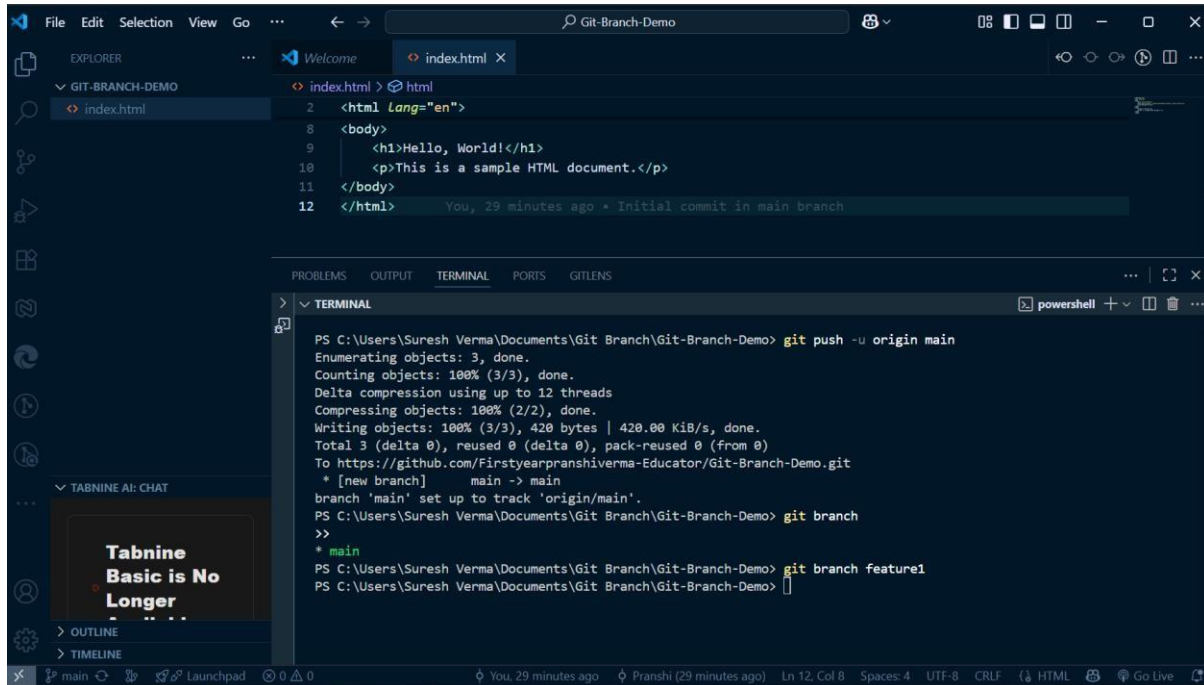
GitHub showing main branch with index.html

Step 7: Check the Current Branch

Output shows all branches. The active branch has a * symbol in front of it (e.g., * main).

Step 8: Create a New Branch

Using the command: `git branch feature1`



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'GIT-BRANCH-DEMO' with a file 'index.html'. The editor pane shows the content of 'index.html', which is an HTML document with a title 'Hello, World!' and a paragraph 'This is a sample HTML document.' The Terminal pane at the bottom shows the following commands and output:

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 420 bytes | 420.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshiverma-Educator/Git-Branch-Demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch
>>
* main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

This creates a new branch named **feature1**.

Step 9: Switch to the New Branch

There are two methods:

Old method (checkout):

`git checkout feature1`

New method (recommended):

`git switch feature1`

```
File Edit Selection View Go ... Git-Branch-Demo
EXPLORER
  GIT-BRANCH-DEMO
    index.html
  TABNINE AI: CHAT
    Tabnine Basic is No Longer
  OUTLINE
  TIMELINE

index.html > html
2 <html Lang="en">
8 <body>
9 <h1>Hello, World!</h1>
10 <p>This is a sample HTML document.</p>
11 </body>
12 </html>

PROBLEMS OUTPUT TERMINAL PORTS GITLENS
TERMINAL
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 420 bytes | 420.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Firstyearpranshivarma-Educator/Git-Branch-Demo.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch
>>
 * main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Now you are working inside the **feature1** branch.

Step 10: Make Changes in index.html

Open your index.html and add one more line under the `<p>` tag. For example:

```
File Edit Selection View Go ... Git-Branch-Demo
EXPLORER
  GIT-BRANCH-DEMO
    index.html M
  TABNINE AI: CHAT
    Tabnine Basic is No Longer
  OUTLINE
  TIMELINE

index.html M > ...
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Document</title>
7 </head>
8 <body>
9 <h1>Hello, World!</h1>
10 <p>This is a sample HTML document.</p>
11 <p>Change made in feature1 branch</p>
12 </body>
13 </html>
14

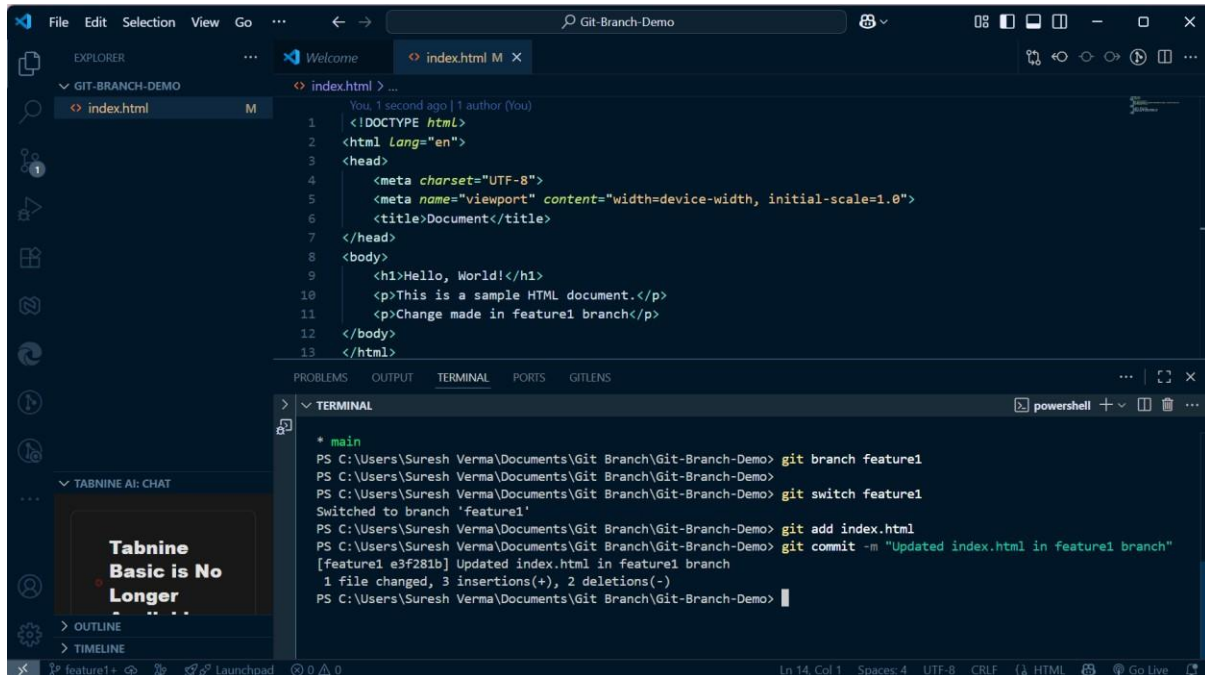
PROBLEMS OUTPUT TERMINAL PORTS GITLENS
TERMINAL
* main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Step 11: Commit the Changes in feature1 Branch

Run these commands:

```
git add index.html
```

```
git commit -m "Updated index.html in feature1 branch"
```



The screenshot shows the Visual Studio Code interface with the 'feature1' branch selected in the Explorer. The file 'index.html' is open in the editor, showing the following content:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>Change made in feature1 branch</p>
12 </body>
13 </html>
```

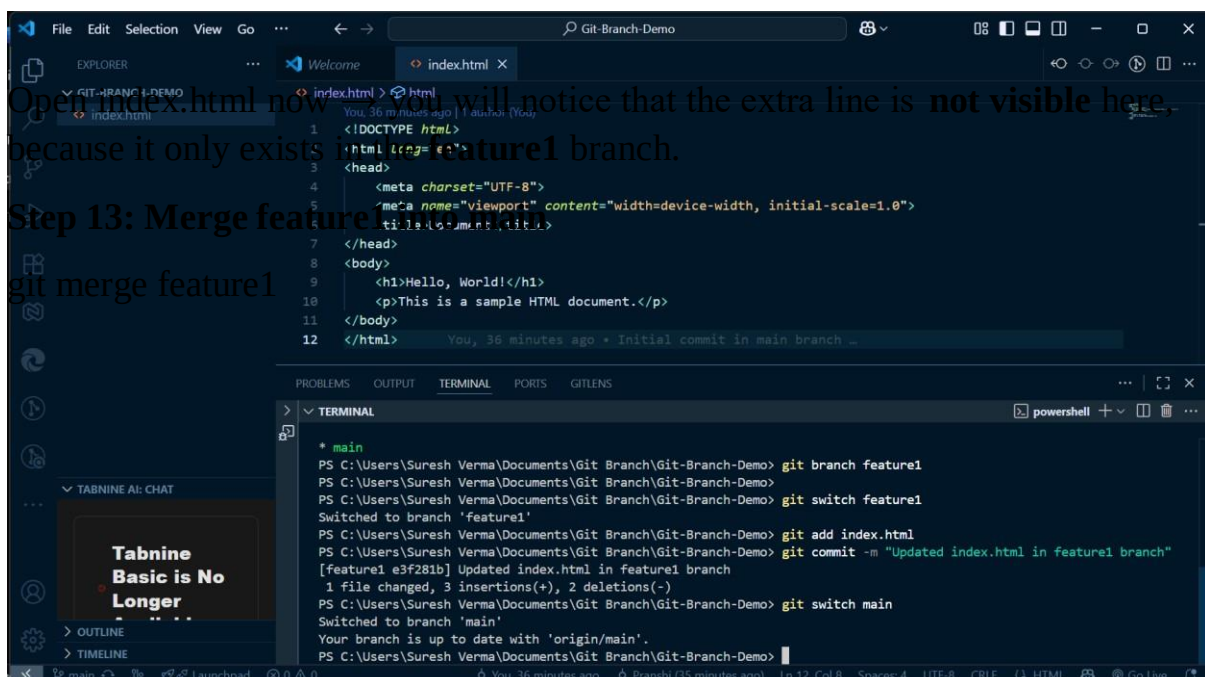
The Terminal window at the bottom shows the following commands and output:

```
* main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Updated index.html in feature1 branch"
[feature1 e3f281b] Updated index.html in feature1 branch
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Commit message for feature1 branch.

Step 12: Switch Back to Main Branch

```
git switch main
```

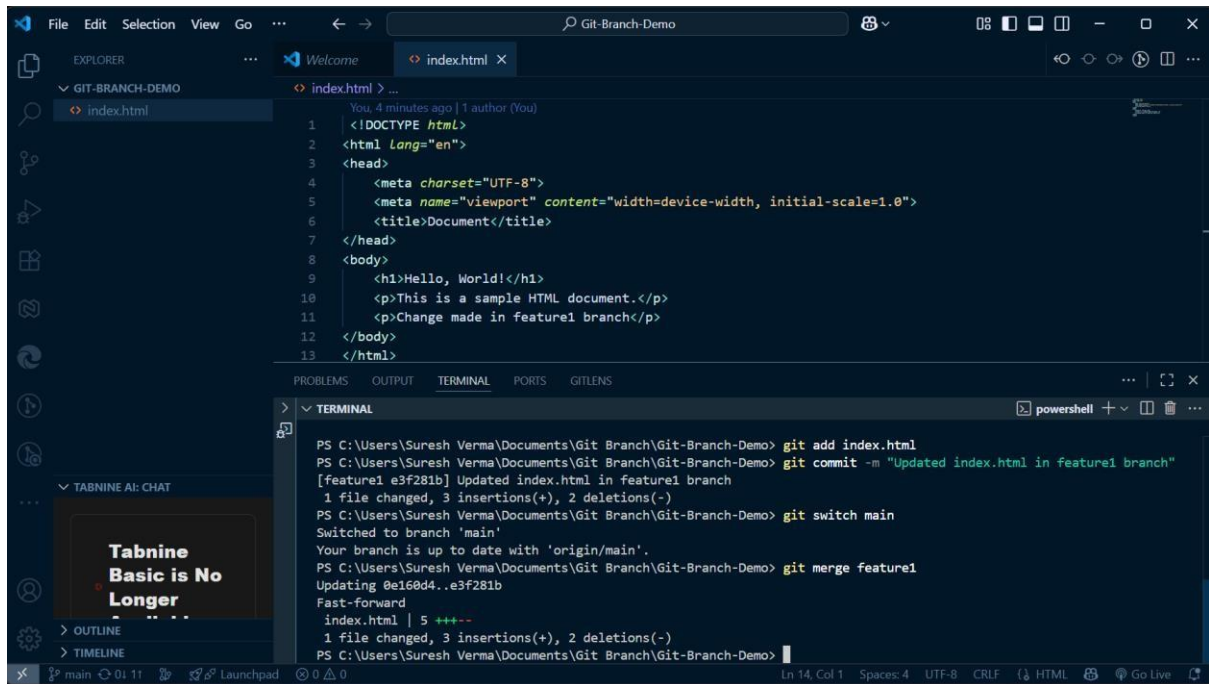


The screenshot shows the Visual Studio Code interface with the 'main' branch selected in the Explorer. The file 'index.html' is open in the editor, showing the following content:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11 </body>
12 </html>
```

The Terminal window at the bottom shows the following commands and output:

```
* main
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git branch feature1
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch feature1
Switched to branch 'feature1'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Updated index.html in feature1 branch"
[feature1 e3f281b] Updated index.html in feature1 branch
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>Change made in feature1 branch</p>
12 </body>
13 </html>
```

```
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Updated index.html in feature1 branch"
[feature1 e3f281b] Updated index.html in feature1 branch
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git merge feature1
Updating 0e168d4..e3f281b
Fast-forward
 index.html | 5 +++--
 1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Now the changes from feature1 are merged into main.
Check index.html again → you will see the new line has appeared.

Step 14 (Optional): Delete the Branch

`git branch -d feature1`

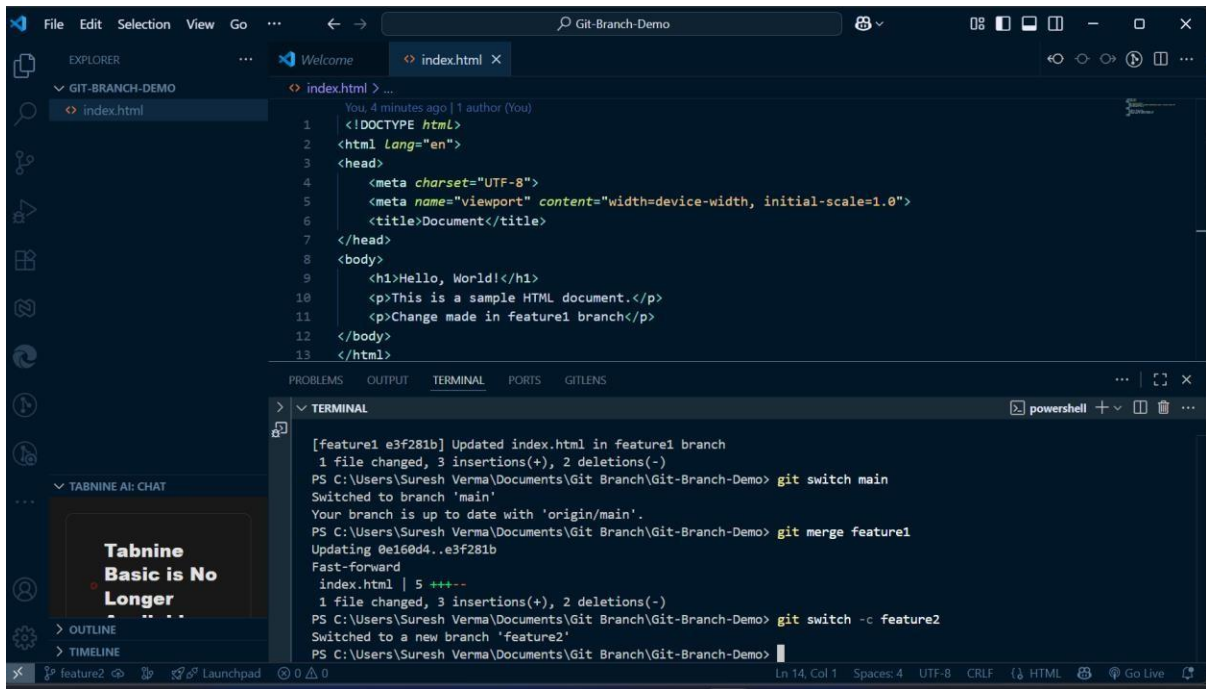
👉 If you don't need the branch anymore, you can delete it safely.

That completes our first **branching + switching + merging** demo.

Step 15 Merge Conflict Demo (with index.html)

From main branch, create a new branch:

`git switch -c feature2`

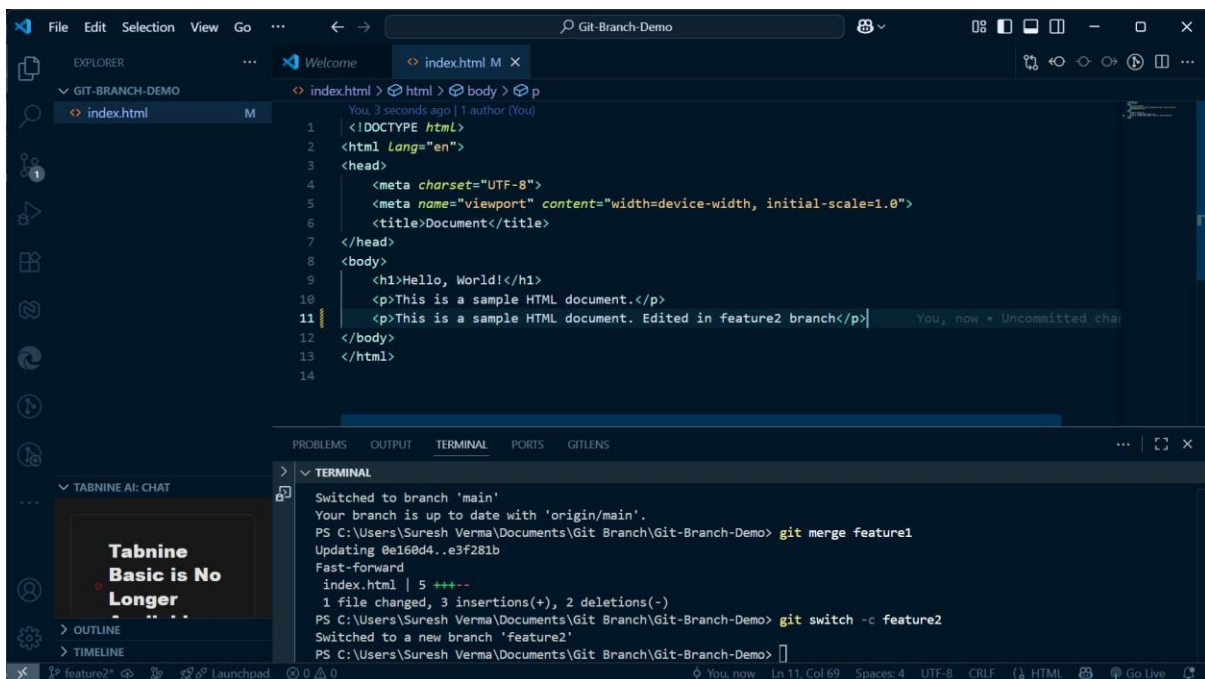


This creates and switches to a new branch **feature2**.

Step 16: Modify index.html in feature2

Change the second `<p>` line like this:

`<p>This is a sample HTML document. Edited in feature2 branch</p>`

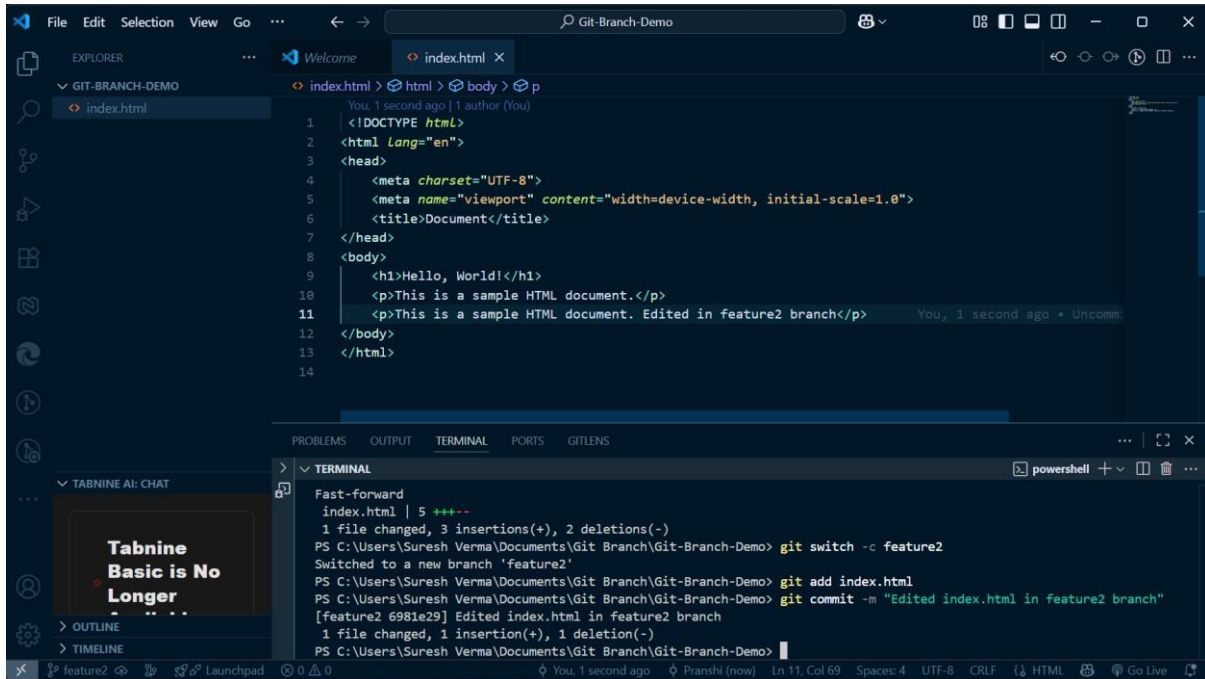


Commit this change:

git add index.html

git commit -m "Edited index.html in feature2 branch"

Commit in feature2 branch



The screenshot shows the Visual Studio Code editor with the 'index.html' file open. The file content is as follows:

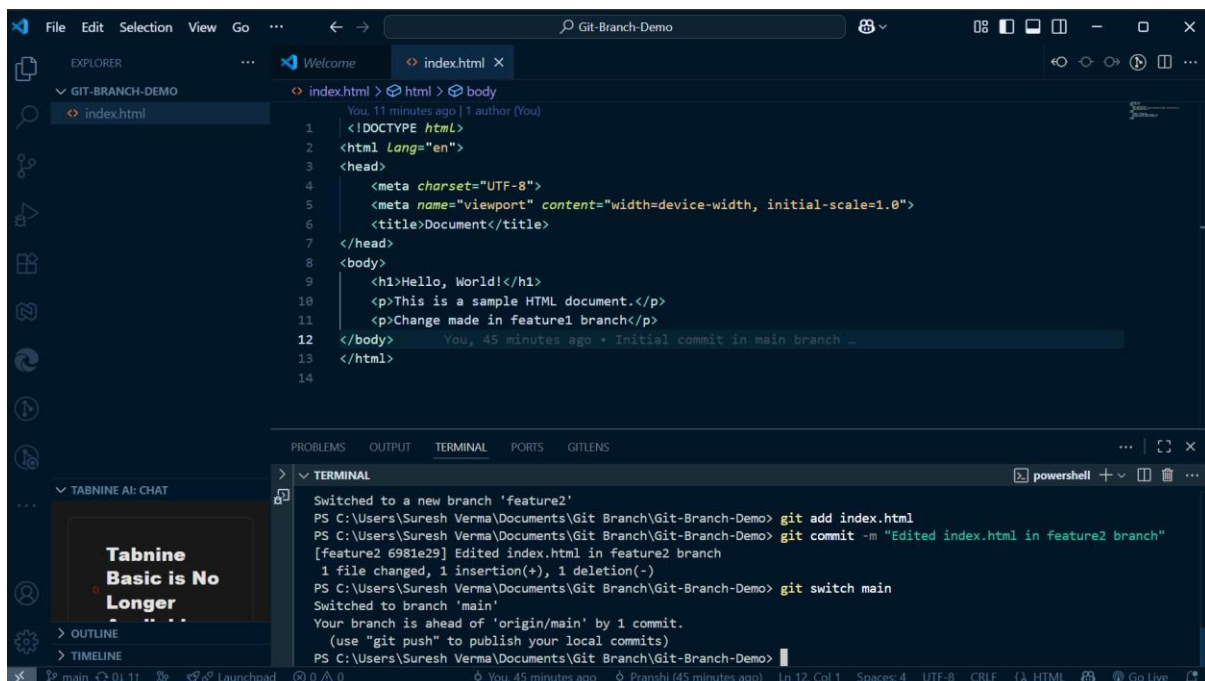
```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>This is a sample HTML document. Edited in feature2 branch</p>
12 </body>
13 </html>
```

The terminal output shows the following commands and their results:

```
Fast-forward
index.html | 5 +++--
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch -c feature2
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in feature2 branch"
[feature2 6981e29] Edited index.html in feature2 branch
1 file changed, 1 insertion(+), 1 deletion(-)
```

Step 17: Switch to main and Make a Different Change

Switch back to main: git switch main



The screenshot shows the Visual Studio Code editor with the 'index.html' file open. The file content is as follows:

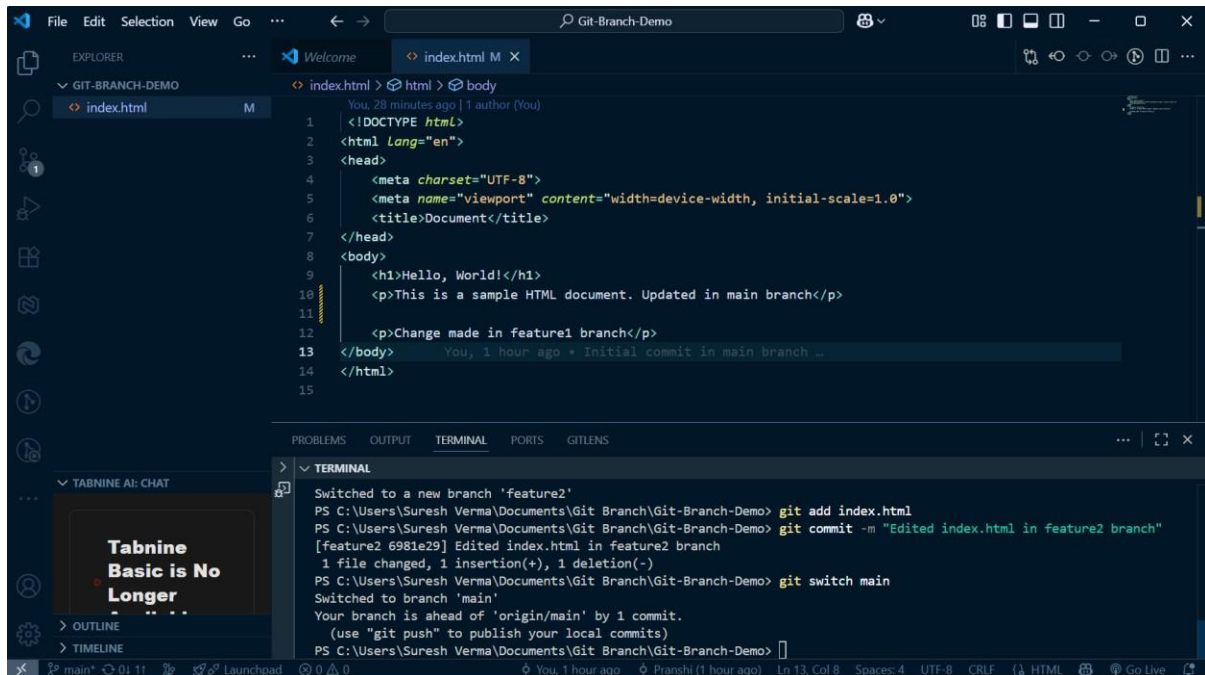
```
1 <!DOCTYPE html>
2 <html Lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document.</p>
11  <p>Change made in feature1 branch</p>
12 </body>
13 </html>
```

The terminal output shows the following commands and their results:

```
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in feature2 branch"
[feature2 6981e29] Edited index.html in feature2 branch
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Now **edit the same line** but write something different:

<p>This is a sample HTML document. Updated in main branch</p>



The screenshot shows the VS Code editor with the file `index.html` open. The file content is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   <h1>Hello, World!</h1>
10  <p>This is a sample HTML document. Updated in main branch</p>
11
12  <p>Change made in feature1 branch</p>
13 </body>
14 </html>
```

The terminal window shows the following commands and output:

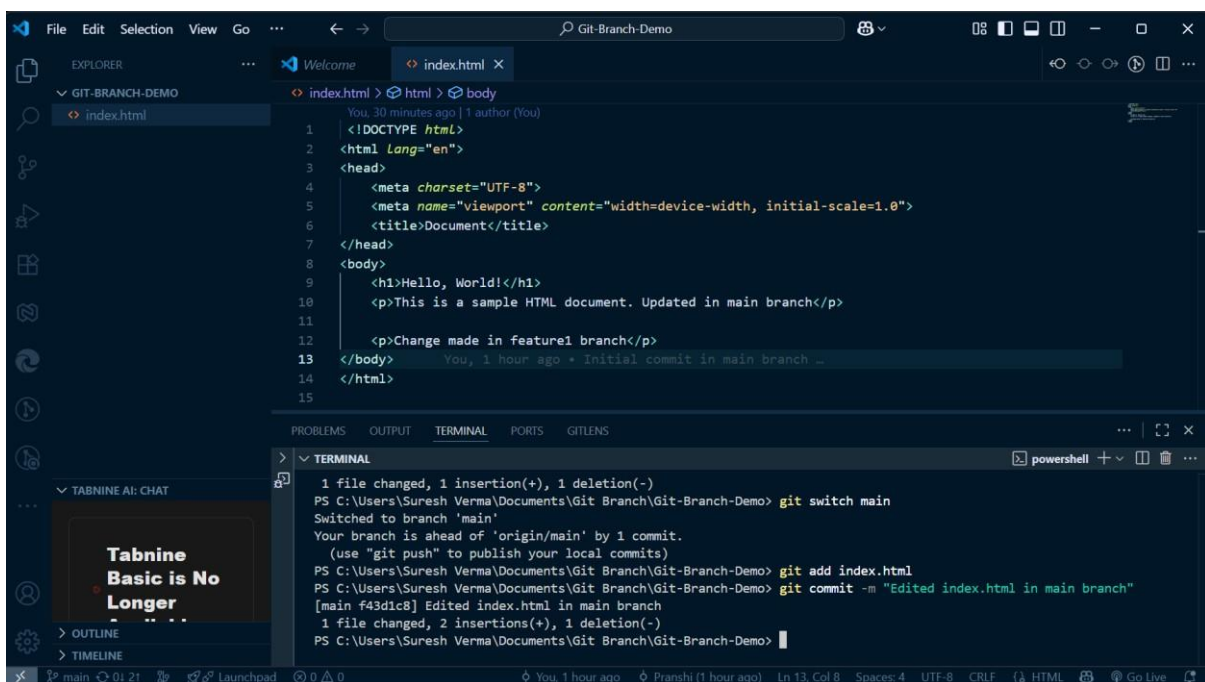
```
Switched to a new branch 'feature2'
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in feature2 branch"
[feature2 6981e29] Edited index.html in feature2 branch
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Commit this change:

`git add index.html`

`git commit -m "Edited index.html in main branch"`

Commit in main branch



The screenshot shows the VS Code editor with the file `index.html` open. The file content is the same as in the previous screenshot. The terminal window shows the following commands and output:

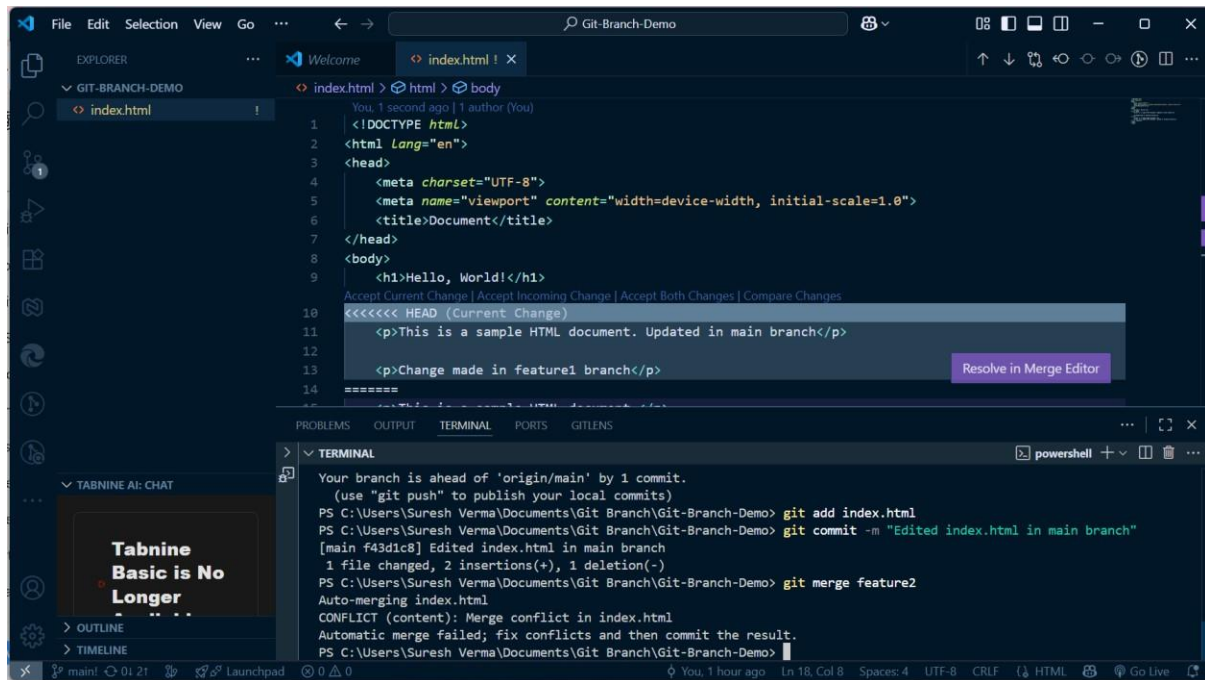
```
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Edited index.html in main branch"
[main f43d1c8] Edited index.html in main branch
1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

Step 18: Try Merging feature2 into main

Now merge:

```
git merge feature2
```

Git will detect that **both branches modified the same line differently** → this creates a **merge conflict**.



Merge conflict message in terminal and The index.html file will show conflict markers like this:

<<<<<<< HEAD → your current branch (main) version

===== → separator

>>>>>>> feature2 → incoming branch version

Step 20: Resolve the Conflict Manually

Decide how you want the final version to look.
For example, keep both lines combined:

```
<body>
```

```
    <h1>Hello, World!</h1>
```

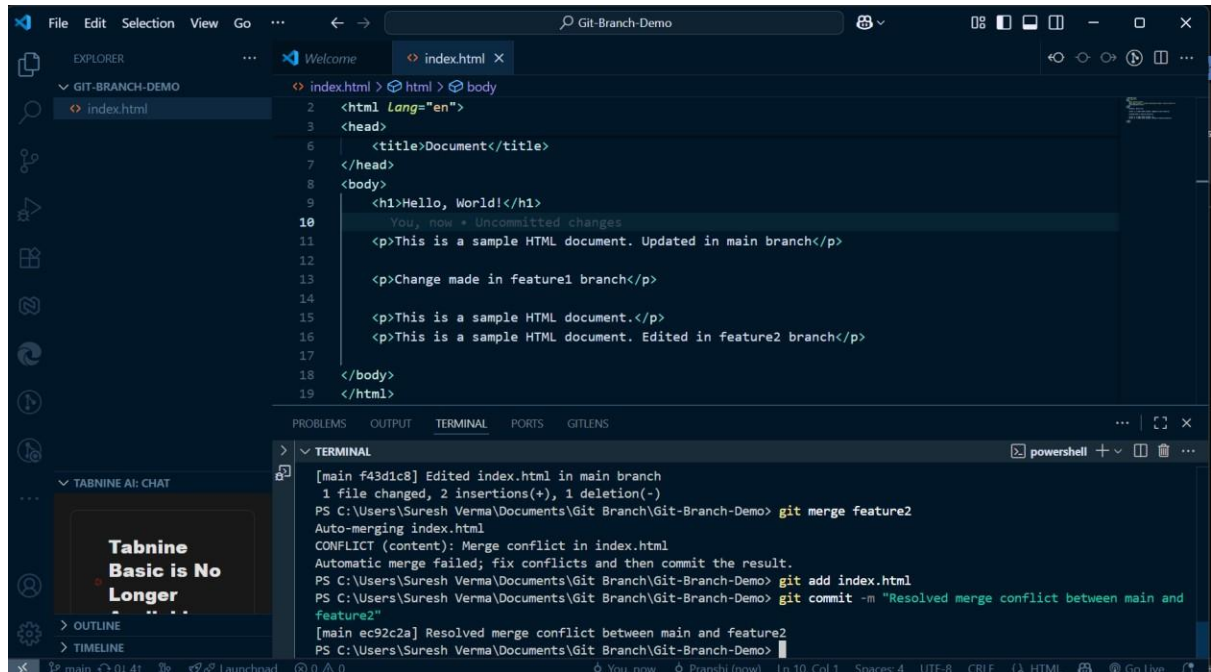
```
    <p>This is a sample HTML document. Updated in main branch & feature2  
    branch</p>
```

<p>Change made in feature1 branch</p>
</body>

Step 21: Mark the Conflict as Resolved

git add index.html

git commit -m "Resolved merge conflict between main and feature2"



```
index.html
1 <html Lang="en">
2 <head>
3   <title>Document</title>
4 </head>
5 <body>
6   <h1>Hello, World!</h1>
7   You, now + Uncommitted changes
8   <p>This is a sample HTML document. Updated in main branch</p>
9   <p>Change made in feature1 branch</p>
10  <p>This is a sample HTML document.</p>
11  <p>This is a sample HTML document. Edited in feature2 branch</p>
12 </body>
13 </html>
```

```
TERMINAL
[main f43d1c8] Edited index.html in main branch
1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git merge feature2
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git add index.html
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo> git commit -m "Resolved merge conflict between main and feature2"
[main ec92c2a] Resolved merge conflict between main and feature2
PS C:\Users\Suresh Verma\Documents\Git Branch\Git-Branch-Demo>
```

- Merge conflicts occur when **two branches edit the same part of a file differently**.
- Git cannot decide automatically, so it asks *you* to resolve.
- You open the file, remove the conflict markers (<<<<<<<, =====, >>>>>>>), and keep the correct content.
- After resolving, you stage and commit the file.

Summary of Key Commands:

- **Create a new branch:** git branch <branch-name>
- **Switch to a branch:** git checkout <branch-name>
- **Create and switch in one step:** git checkout -b <branch-name>
- **Stage changes:** git add <file-name> or git add .
- **Commit changes:** git commit -m "message"

- **List all branches:** `git branch`
 - **Merge a branch into the current branch:** `git merge <branch-name>`
 - **Delete a branch:** `git branch -d <branch-name>`
 - **Push changes:** `git push origin <branch-name>`
-

3.4 GitHub collaboration workflow, Best practices for commits, branches, and messages

3.4.0 Collaboration Workflow in GitHub

When working in a **team project**, multiple developers collaborate. The most common workflow is:

Step 1: Fork (for open-source projects) / Clone (for team repos)

- If you're contributing to someone else's repository → **Fork** it into your account.
- If you're in a team and already have access → **Clone** the repo.

Forked repo (on GitHub) → then clone it

```
git clone https://github.com/your-username/project.git
```

```
cd project
```

(Already discussed with eg.)

Step 2: Create a New Branch for Your Work

Never work directly on the main branch.

```
git checkout -b feature/login-page
```

Real-life example:

- Project repo = **E-commerce website**
- You → creating **login page** feature → branch name = `feature/login-page`

Step 3: Make Changes Locally

- Edit files, add new features, or fix bugs.
- Stage changes:

```
git add .
```

- Commit with a meaningful message:

```
git commit -m "Added login form with validation"
```

Step 4: Push the Branch to GitHub

```
git push origin feature/login-page
```

Now your branch is visible on GitHub.

Step 5: Create a Pull Request (PR)

- Go to GitHub → Open a **Pull Request** from feature/login-page → main.
- Describe what you did.
- Example PR message:

"Implemented login page with form validation and session handling."

Step 6: Code Review & Discussion

- Teammates review your code.
 - They can request **changes** or **approve**.
-

Step 7: Merge PR

- Once approved → merge into main.
- If conflicts appear → resolve with:

```
git pull origin main
```

```
# Fix conflicts in files
```

```
git add .
```

```
git commit -m "Resolved merge conflicts"
```

```
git push origin feature/login-page
```

Step 8: Delete Branch (after merge)

```
git branch -d feature/login-page    # local
```

```
git push origin --delete feature/login-page  # remote
```

3.4.1 Best Practices for Commits, Branches, and Messages

A. Branching Best Practices

- Use **descriptive names**:
 - feature/login-page
 - bugfix/cart-issue
 - hotfix/security-patch
 - One branch = One task/feature
-

B. Commit Best Practices

- Small, frequent commits (atomic commits).
 - Avoid large commits like: "updated everything".
 - Good messages:
 - "Added search functionality in product list"
 - "Fixed bug in checkout page discount calculation"
 - Bad messages:
 - "Fixed", "Update code", "Work done"
-

C. Pull Request Best Practices

- Clear **title + description**.
 - Link issue number (if using GitHub issues).
 - Request review from teammates.
-

D. General Workflow Best Practices

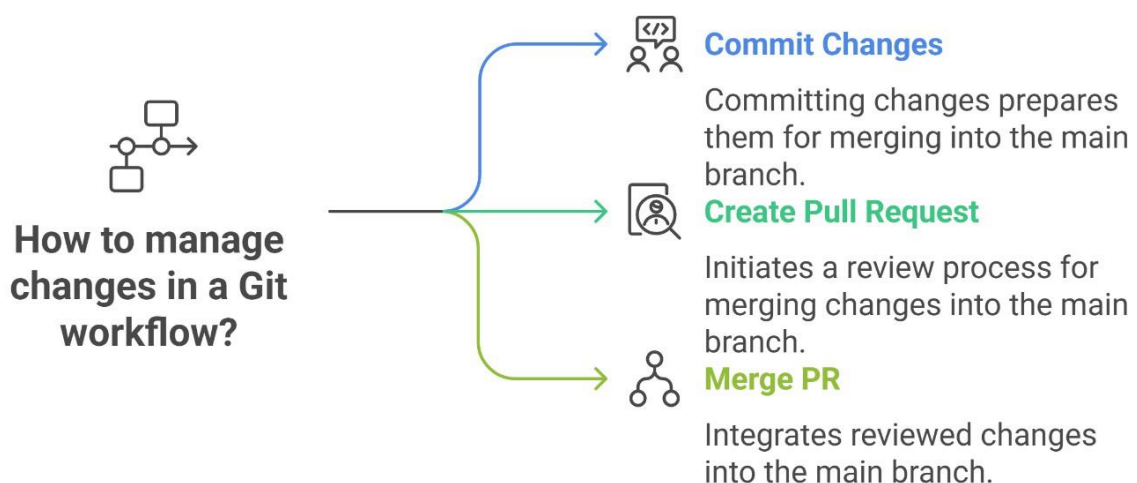
1. **Always pull latest changes before starting work:**
 2. `git pull origin main`
 3. **Work in your own branch.**
 4. **Keep commits clean.**
 5. **Write meaningful PR descriptions.**
 6. **Delete merged branches** to avoid clutter.
-

3.4.2 Real-Life Example Flow

Team Project: *E-commerce Website*

- **Alice** creates feature/login-page.
 - **Bob** creates feature/cart-system.
 - Both work separately.
 - They push branches → make PR → merge into main.
 - The main branch always has **stable, production-ready code**.
-

3.4.3 Diagram of GitHub Collaboration Workflow



Made with Napkin

This workflow ensures **smooth collaboration**, prevents conflicts, and keeps the main branch stable.
